(1)The Role of Algorithms in Computing + Nature & Size of Input

The Role of Algorithms in Computing:

- The computer is just a tool. The algorithm is the intelligence. A faster computer running a bad algorithm will almost always lose to a slower computer running a good one.
- A good algorithm can give you a speedup of millions or billions. No hardware improvement ever comes close to that.
- Algorithms are what make computation meaningful. Without algorithms, a computer is just electricity moving through silicon.

Algorithms as a Technology:

Algorithms are a technology, just like hardware, just like operating systems, just like programming languages.

Suppose you are sorting one million numbers.

Insertion Sort takes roughly n^2 operations in the worst case.

For n = 1,000,000, that is 1 trillion operations.

Merge Sort takes roughly $n \log_2 n$ operations.

For n = 1,000,000, that is roughly 20 million operations.

The ratio is 1,000,000,000,000 versus 20,000,000. That is a 50,000×difference from the algorithm alone, before we even think about hardware.

Algorithms and Hardware:

- Hardware improvements and algorithmic improvements don't just add, they multiply.
- Suppose a machine runs 10 million operations per second, and it is upgraded to one that runs 100 million. That's a 10× improvement. Now suppose we also swap our $O(n^2)$ algorithm for an O(n log n) one. For large n, that improvement is much more than 10×.
- The combined effect of better hardware and a better algorithm is multiplicative you get both benefits simultaneously.

The Algorithm as a Bridge:

- Think of an algorithm as a bridge between a problem statement and a computational solution.
- On one side, there is a well-defined problem sort these numbers, find the shortest path, determine if this number is prime.
- On the other side, we have a machine that can execute basic operations: compare, add and move data.
- The algorithm is the precise sequence of those basic operations that transforms the problem input into the correct output.

Input Size:

When we write T(n) to denote the running time of an algorithm, n is the input size. But input size is not always as straightforward as it sounds. The right definition of n depends on the problem.

For sorting and searching: n is the number of elements in the array or list. If you are sorting 10,000 numbers, n = 10,000.

For graph algorithms: A graph has both vertices and edges. The running time of a graph algorithm usually depends on both. We typically use n for the number of vertices and m/e for the number of edges. An algorithm might run in O(n + m) time meaning its work grows with both the number of nodes and the number of edges.

For arithmetic on integers: if we are computing whether a large number is prime, the natural

Measure of size is not the value of the number, but the number of digits which is $\log_{10}(n)$, or in binary, the number of bits.

For matrix operations: n is typically the dimension. An n × n matrix has n^2 entries, so even reading every entry is an $O(n^2)$ operation.

Always be explicit about what n means in your problem. Different choices of n can make the same algorithm look either efficient or inefficient.

The Input Size Determines the Trajectory:

Algorithm	n = 10	n = 100	n = 1,000	n = 1,000,000
O(log n)	~3 ops	~7 ops	~10 ops	~20 ops
O(n)	10 ops	100 ops	1,000 ops	1,000,000 ops
O(n log n)	~33 ops	~664 ops	~10,000 ops	~20,000,000 ops
$O(n^2)$	100 ops	10,000 ops	1,000,000 ops	10^{12} ops
$O(2^n)$	1,024 ops	10^{30} ops	10^{301} ops	incomprehensible

The Input Size Determines the Trajectory

Look at what happens to $O(n^2)$ as n goes from 1,000 to 1,000,000. It goes from one million operations to one trillion. That's a factor of one million increase in work for a factor of one thousand increase in input

size. The work grows as the square of the input growth.

Now look at O(log n). Going from n = 10 to n = 1,000,000 a factor of 100,000 increase in input only adds about 17 operations. This is the magic of logarithmic algorithms. They are almost indifferent to input size.

These trajectories are what asymptotic notation is designed to capture precisely.

<u>The Nature of Input</u>:

Input size is not the whole story. Two inputs of the same size can cause the same algorithm to do vastly different amounts of work.

This is the nature of input, and it introduces the three most important concepts in algorithm analysis:

• Best case

• Worst case

• Average case.

Consider **Linear Search:** we have an unsorted array of n elements, and we want to find whether a specific value x is in the array.

Linear Search(A, n, x):

for i = 1 to n:

if A[i] == x:

return i ← found it

return -1 ← not found

How many comparisons does this algorithm make?

Best Case:

The best case is when the element we are looking for is in the first position of the array. We check A[1], it matches, and we return immediately. We did exactly 1 comparison, regardless of how large n is.

Best case running time for Linear Search: T(n) = 1.

That is O(1) constant time.

Does this mean Linear Search is a fast algorithm? Absolutely not. The best case is a degenerate, unrealistic scenario. In real usage, we cannot count on the element always being first. Best case analysis in isolation is almost always misleading.

However, best case does have a legitimate use: it tells us the theoretical lower bound on the work the algorithm will ever do.

No execution of Linear Search will ever do fewer than 1 comparison.

Worst Case: The worst case is when the algorithm has to do the maximum possible work.

For Linear Search, this happens in two situations:

The element x is the last element in the array we check all n elements before finding it.

The element x is not in the array at all we check all n elements and never find it.

In both situations, we make exactly n comparisons.

Worst case running time for Linear Search: T(n) = n.

That is O(n) — linear time.

Average Case: The average case tries to answer: on a typical input, how does the algorithm perform?

For Linear Search, let's assume two things: the element x is present in the array, and it is equally likely to be in any of the n positions. That is a uniform distribution assumption.

If x is equally likely to be at position 1, 2, 3, ..., or n, then:

If x is at position 1, we make 1 comparison.

If x is at position 2, we make 2 comparisons.

If x is at position k, we make k comparisons.

If x is at position n, we make n comparisons.

The average number of comparisons is:

$(1 + 2 + 3 + ... + n) / n = (n(n+1)/2) / n = (n+1)/2$

So on average, we examine about half the array roughly n/2 comparisons.

That is still O(n) the same asymptotic complexity as the worst case. But the constant factor is half as large in practice.

For many algorithms, best, average, and worst case all have the same asymptotic complexity.

Space Complexity:

The **space complexity** of an algorithm is the amount of memory it uses as a function of input size n. This includes:

- The **memory** used to store the input itself (though sometimes we don't count this, depending on the model).
- Any additional memory allocated during execution auxiliary arrays, recursive call stacks, temporary variables.
- Time and space are often in tension**. Algorithms** that are very fast in time may require significant memory. Algorithms that use minimal memory may be slower. Understanding both dimensions is part of what it means to fully analyze an algorithm.

(2) Asymptotic Notations

Complexity:

Complexity is about resource consumption and there are two resources we care about:

Time complexity: how many elementary operations does the algorithm perform?

We model this as a count of basic steps: comparisons, assignments, arithmetic operations. We treat each elementary operation as taking one unit of time. This is

an abstraction real operations take different amounts of time on real hardware but it is a clean and useful model.

Space complexity: how much memory does the algorithm use beyond its input?

Every variable, every array, every recursive call stack frame costs memory. Space complexity measures that cost as a function of n.

In both cases the question is the same: as n grows, how does the cost grow? Does it stay flat? Does it double every time n doubles? Does it square? Does it explode exponentially? The answer to that question is the complexity of the algorithm.

What Does Asymptotic Mean?

The word asymptotic comes from mathematics. An asymptote is a line or curve that another curve approaches but never quite reaches as you go to infinity. When we say asymptotic analysis, we mean: we are studying the behavior of functions as n approaches infinity.

Why infinity? Because we want to capture the long-run behavior what happens when inputs get large. For small inputs, almost any algorithm is fast enough. The interesting, practically relevant question is: what happens when n is ten thousand, a million, a billion? That is where

algorithmic choices become life-or-death for a system.

Asymptotic Notations

Asymptotic notations are a mathematical language for describing the growth rates of functions. They give us a standardized, precise vocabulary for making statements like:

"This algorithm's running time grows no faster than n squared" Big-O

"This algorithm's running time grows no slower than n log n" Big-Ω

"This algorithm's running time grows at exactly the same rate as n log n, up to constant factors" Big-Θ

Asymptotic analysis focuses on the rate of growth of an algorithm's runtime or space requirements.

Instead of measuring exact seconds (which vary by hardware), we use functions to describe how the number of operations increases with input size

The core philosophy of asymptotic analysis:

drop lower-order terms, drop constant factors, keep the dominant growth pattern.

$$\mathbf{T(n) = 4n^2 + 7n + 23}$$

operations on an input of size n. Now let's ask: for large n, which term matters?

At n = 1,000

$4n^2 = 4{,}000{,}000$

$7n = 7{,}000$

$23 = 23$

The 7n term is 0.17% of the total. The constant 23 is essentially zero. The n^2 term is everything.

At n = 1,000,000, the 7n and 23 terms are even more negligible. As n grows toward infinity, the n^2 term completely dominates the other terms become rounding errors.

Asymptotic Notations Properties

• Categorize algorithms based on asymptotic growth rate e.g. linear, quadratic, polynomial, exponential.

• Ignore small constant and small inputs.

• Estimate upper bound and lower bound on growth rate of time complexity function.

• Describe running time of algorithm as n grows to.

• Describes behavior of function within the limit.

Limitations:

• Not always useful for analysis on fixed-size inputs.

• All results are for sufficiently large inputs.

Growth Rate of Functions:

The Growth Rate (or Order of Growth) defines how the running time of an algorithm increases as the input size n becomes very large.

It is the most critical factor in comparing algorithm efficiency because hardware-specific constants become irrelevant at scale.

We evaluate performance as $n \to \infty$.

An algorithm with a slower growth rate is considered superior for large datasets.

Big-O Notation:

The upper bound

We say $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that:

$0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

$f(n) = O(g(n))$ means function g(n) is an asymptotically upper bound for f(n).

We may write $f(n) = O(g(n))$ OR $f(n) \in O(g(n))$

Set of all functions whose rate of growth is the same as or lower than that of g(n).

Big-Omega Notation (Ω) :

The Lower Bound

$f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that:

$0 \leq f(n) \geq c \cdot g(n)$ for all $n \geq n_0$

$f(n) = \Omega\ (g(n))$ means function g(n) is an asymptotically lower bound for f(n).

We may write $f(n) = \Omega(g(n))$ OR $f(n) \in \Omega(g(n))$

Set of all functions whose rate of growth is the same as or higher than that of g(n).

Big-Theta Notation(Θ):

The Tight Bound Big-Θ is where both Big-O and Big-Ω come together. It provides a tight bound a precise characterization of growth rate.

$f(n) = \Theta(g(n))$ if and only if:

$f(n) = O(g(n))$ AND $f(n) = \Omega(g(n))$

Expanding this out: there exist positive constants c_1, c_2, and n_0 such that:

$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$

f(n) is sandwiched between two constant multiples of g(n) forever beyond n_0. The function g(n) is both a ceiling and a floor for f(n) up to constant factors.

$f(n) = \Theta(g(n))$ means function f(n) is equal to g(n) within a constant factor and g(n) is asymptotically tight bound for f(n).

We may write $f(n) = \Theta(g(n))$ OR $f(n) \in \Theta(g(n))$

Set of all functions that have same rate of growth as g(n).

Problem(Big O Notation)

Find the Upper Bound for f(n)=2n2+5n+80

We must find positive constants 'c' and 'n0'

such that: f(n) <= c * g(n) for all n >= n0

f(n)=2n2+5n+80,the highest power of n is n2. Therefore, our target g(n)=n2.

2n2+ 5n + 80 <= 2n2 + 5n2 + 80n2

2n2+ 5n + 80 <= 87n2

for n = 1: 2(1)2+ 5(1) + 80 <= 87(1)2

87 <= 87 is true, we can use: n0= 1.

Since there exist constants c=87andn0=1suchthat

f(n) <= c * n2for all n >= n0,

we conclude: f(n) = O(n2)

Problem(2) Big O Notation

f(n) = 5n + 50 g(n) = n

is g(n) upper bound of f(n)?

We must find positive constants 'c' and 'n0'

such that: f(n) <= c * g(n) for all n >= n0

f(n) = 5n + 50, the highest power of n is n.

5n + 50 <= 5n + 50n

5n + 50 <= 55n

For n = 1: 5(1) + 50 <= 55(1)

55 <= 55 is true, we can use: n0 = 1.

Since there exist constants c = 55 and n0 = 1

such that f(n) <= c * n for all n >= n0,

we conclude: f(n) = O(n) Yes, g(n) = n is the upper bound of f(n).

Problem(3)Big O Notation

Prove that $2n2 \in O(n3)$

Proof

: Assume that f(n) = 2n2, and g(n) = n3

Now we have to find the existence of c and n0

such that: $f(n) \leq c * g(n)$

$2n2 \leq c * n3$

Divide both sides by n2: $2 \leq c * n$

If we take: c = 1 and n0 = 2 (Since $2 \leq 1 * 2$ is true)

OR

c = 2 and n0 = 1 (Since $2 \leq 2 * 1$ is true)

Then: $2n2 \leq c * n3$

Hence $f(n) \in O(g(n))$ for c = 2 and n0 = 1.

<u>Little o (Strict Upper Bound)</u>

Big-O allows f and g to grow at the same rate.

little-o does not- it requires f to grow strictly slower than g.

Formal Definition:

$f(n) = o(g(n))$ if for every positive constant c, there exists n_0 such that:

$f(n) < c \cdot g(n)$ for all $n \geq n_0$

Notice the critical difference from Big-O. In Big-O, we said "there exists a c." In little-o, we say "for every c." This is a fundamentally stronger requirement. We need f(n) to be dominated by g(n) no matter how small we make the constant c. g(n) dominates f(n) so completely that even c = 0.00001 is eventually enough to keep g above f.

The limit characterization:

$f(n) = o(g(n))$ if and only if $\lim[n\to\infty] f(n)/g(n) = 0$

The ratio of f to g goes to zero. f becomes negligible compared to g as n grows.

Little ω (Strict Lower Bound)

little-ω is to Big-Ω what little-o is to Big-O. It is the strict version of the lower bound.

Formal Definition:

$f(n) = \omega(g(n))$ if for every positive constant c, there exists n_0 such that:

$f(n) > c \cdot g(n)$ for all $n \geq n_0$

The limit characterization:

$f(n) = \omega(g(n))$ if and only if $\lim[n\to\infty] f(n)/g(n) = \infty$

The ratio of f to g blows up to infinity. f dominates g so completely that no constant multiple of g can keep up with f.

Limit Characterization:

Limit characterization is often called the Calculus way to determine asymptotic notation. Instead of hunting for constants c and n0 we look at the ratio of two functions as n goes to infinity.

It is much faster for complex functions because you can use L'Hôpital's Rule.

We define the limit as:

$L = \lim[n \to \infty] f(n)/g(n)$

$$L = \lim_{n \to \infty} \frac{f(n)}{g(n)}$$

Depending on the value of L, we can determine the relationship:

If the Limit L is...	Then the relation is...	Intuition
0	$f(n) = o(g(n))$ and $O(g(n))$	f grows much slower than g.
A constant $c > 0$	$f(n) = \Theta(g(n))$	f and g grow at the same rate.
∞	$f(n) = \omega(g(n))$ and $\Omega(g(n))$	f grows much faster than g.

If L is a constant c > 0 then f(n) is also O(g(n)) and Ω(g(n)) because Big Theta implies both

Limit Characterization

L'Hôpital's Rule

When we get an indeterminate form like ∞/∞ or 0/0 we will take the derivative of the top and bottom separately:

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = \lim_{n \to \infty} \frac{f'(n)}{g'(n)}$$

Problem: Compare f(n) = 3n2 + 5n and g(n) = n2.

$L = \lim (n \to \infty) [(3n^2 + 5n) / n^2]$

Divide terms by n^2:

$L = \lim (n \to \infty) [3 + 5/n]$

$L = 3 + 0 = 3$

Result: $f(n) = \Theta(n^2)$ because the limit is a positive constant.

Problem(2): Compare f(n) = 100n and g(n) = n2.

$L = \lim (n \to \infty) [100n / n^2]$

Simplify the fraction:

$L = \lim (n \to \infty) [100 / n]$

$L = 100 / \infty = 0$

Result: $f(n) = o(n^2)$ because the limit is 0 (f grows slower than g).

Problem(3): Compare f(n) = n2 and g(n) = ln(n).

$L = \lim (n \to \infty) [n^2 / \ln(n)] \longrightarrow$ (Form: ∞/∞)

Apply L'Hôpital's Rule (Derivatives):

Derivative of n^2 is 2n

Derivative of ln(n) is 1/n

$L = \lim (n \to \infty) [2n / (1/n)]$

$L = \lim (n \to \infty) [2n^2]$

$L = 2(\infty)^2 = \infty$

Result: $f(n) = \omega(\ln n)$ because the limit is ∞ (f grows faster than g).

Relations Over Asymptotic Notations

Reflexive Relation:

Let X be a non-empty set and R is a relation over X then R is said to be reflexive if $(a, a) \in R, \forall a \in X$,

Example 1: Let G be a graph. Let us define a relation R over G as if node

x is connected to y then $(x, y) \in G$.

Reflexivity is satisfied over G if for every node there is a self loop.

Example 2: Let P be a set of all persons, and S be a relation over P such that if $(x, y) \in S$ then x has same birthday as y. Of course this relation is reflexive because $(x, x) \in S, \forall a \in P$,

Reflexivity:

$f(n) = O(f(n))$ every function is Big-O of itself

Since, $0 \leq f(n) \leq cf(n) \quad \forall n \geq n0 = 1$, if c = 1

$f(n) = \Omega(f(n))$ every function is Big-Ω of itself

Since, $0 \leq f(n) \geq cf(n) \quad \forall n \geq n0 = 1$, if c = 1

$f(n) = \Theta(f(n))$ every function is Big-Θ of itself

Since, $0 \leq c1f(n) \leq f(n) \leq c2f(n) \quad \forall n \geq n0 = 1$,if c1= c2 = 1

Little o and ω are not Reflexivity Relations

Example

As we can not prove that $f(n) < f(n)$, for any n, and for all $c > 0$

Therefore

1. $f(n) \neq o(f(n))$

2. $f(n) \neq \omega(f(n))$

Hence small o and small omega are not reflexive relations

Symmetry

Let X be a non-empty set and R is a relation over X then R is said to be

symmetric if $\forall$ a, b $\in$ X, (a, b) $\in$ R $\Longrightarrow$(b, a) $\in$ R

Example 1:

Let P be a set of persons, and S be a relation over P such that if (x, y) $\in$ S then x has the same sign as y.

This relation is symmetric because (x, y) $\in$ S $\Longrightarrow$ (y, x) $\in$ S

Example 2:

Let P be a set of all persons, and B be a relation over P such that if (x, y) $\in$ B then x is brother of y.

This relation is not symmetric because (Anwer, Sadia) $\in$ B $\Longrightarrow$(Sadia, Brother) $\notin$ B

Symmetry over Θ Property:

Prove that $f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$

Proof:

By definition of Θ, if $f(n) = \Theta(g(n))$, there exist constants c1, c2 > 0:

$$0 \leq c1*g(n) \leq f(n) \leq c2*g(n) \text{ for all } n \geq n0$$

From the right side

$$(f(n) \leq c2*g(n)):$$

Divide by c2:

$$(1/c2)*f(n) \leq g(n)$$

From the left side

$$(c1*g(n) \leq f(n))$$

Divide by c1:

$$g(n) \leq (1/c1)*f(n)$$

Combining these results:

$$0 \leq (1/c2)*f(n) \leq g(n) \leq (1/c1)*f(n)$$

Let c3 = 1/c2 and c4 = 1/c1:

$$0 \leq c3*f(n) \leq g(n) \leq c4*f(n)$$

This matches the definition of Θ:

$g(n) = \Theta(f(n))$Hence, $f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$

Transitivity:

Let X be a non-empty set and R is a relation over X then R is said to be transitive if $\forall$ a, b, c $\in$ X, (a, b) $\in$ R $\wedge$ (b, c) $\in$ R $\Longrightarrow$ (a, c) $\in$ R

Example 1: Let P be a set of all persons, and B be a relation over P such that if (x, y) $\in$ B then x is brother of y.

This relation is transitive this is because (x, y) $\in$ B $\wedge$ (y, z) $\in$ B $\Longrightarrow$ (x, z) $\in$ B.

Example 2: Let P be a set of all persons, and F be a relation over P such that if (x, y) $\in$ F then x is father of y.

Of course this relation is not a transitive because

if (x, y) $\in$ F $\wedge$ (y, z) $\in$ F $\Longrightarrow$ (x, z) $\notin$ F

If f = O(g) and g = O(h), then f = O(h)

If f = Ω(g) and g = Ω(h), then f = Ω(h)

If f = Θ(g) and g = Θ(h), then f = Θ(h)

Same holds for little-o and little-ω

Transitivity lets us chain complexity comparisons, which will be used constantly when reasoning about composed algorithms.

Transpose symmetry:

$f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$

$f(n) = o(g(n))$ if and only if $g(n) = \omega(f(n))$

This one is deeply intuitive. If f is an upper bound on g, then g is a lower bound on f. If f grows strictly slower than g, then g grows strictly faster than f. The direction flips when you flip the relationship.

Prove that $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$

Proof

Since $f(n) = O(g(n)) \Rightarrow \exists$ constants $c > 0$ and $n0 \in N$

such that $0 \leq f(n) \leq cg(n) \quad \forall\ n \geq n0$

Dividing both sides by c:

$0 \leq (1/c)f(n) \leq g(n) \quad \forall\ n \geq n0$

Putting $1/c = c'$:

$0 \leq c'f(n) \leq g(n) \quad \forall\ n \geq n0$

Hence, $g(n) = \Omega(f(n))$

Prove that $f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f(n))$

Proof

Since $f(n) = o(g(n)) \Rightarrow \exists$ constants $c > 0$ and $n0 \in N$

such that $0 \leq f(n) < cg(n) \quad \forall\ n \geq n0$

Dividing both sides by c:

$0 \leq (1/c)f(n) < g(n) \quad \forall\ n \geq n0$

Putting $1/c = c'$:

$0 \leq c'f(n) < g(n) \quad \forall\ n \geq n0$

Hence, $g(n) = \omega(f(n))$

Big-O is not a tight bound. Neither is Big Ω:

When an algorithm runs in $\Theta(n \log n)$ time, all of the following are

technically true:

It is	$O(n \log n)$	correct and tight
It is	$O(n^2)$	correct but loose
It is	$O(n^{100})$	correct but absurdly loose
It is	$\Omega(n)$	correct but loose
It is	$\Omega(\log n)$	correct but very loose

None of those loose bounds are wrong. They are just not useful. The reason we care about tight bounds Θ is that they give the most information.

A loose Big-O tells something. A tight Θ tells everything about growth behavior.

When someone says an algorithm is $O(f(n))$, they usually mean it as a tight bound even if they are technically only stating an upper bound.

The Standard Complexity Classes

O(1) — Constant time: The algorithm does the same amount of work regardless of input size. Array index access, hash table lookup (average case).

O(log n) — Logarithmic time: The work grows logarithmically. Binary Search is the canonical example.

O(n) — Linear time: Work grows proportionally with input. Linear Search, finding the maximum in an unsorted array.

O(n log n) — Linearithmic time: The sweet spot for many important problems. Merge Sort, Heap Sort, and many other optimal algorithms live here.

$O(n^2)$ — Quadratic time: Work grows as the square of input size. Bubble Sort, Insertion Sort (worst case), simple graph algorithms.

$O(n^3)$ — Cubic time: Naive matrix multiplication. Only practical for small to medium inputs.

$O(2^n)$ — Exponential time: Grows catastrophically. Exact solutions to many NP-hard problems.

O(n!) — Factorial time: Brute-force combinatorial search.

Chp3 Loop Invariants

Loop Invariant

A loop invariant is a property or condition that holds true before, during, and after every iteration of a loop. It's a logical statement about the state of the program that can be used to formally prove that the algorithm does what is claimed.

Think of it like a contract: before the loop runs, the contract is in place.

Each iteration of the loop must maintain the contract. When the loop ends, the contract (combined with the fact that the loop terminated) gives you the proof of correctness.

The Three Obligations

We use loop invariant to help us understand why an algorithm is correct.

We must show three things about a loop invariant:

1. Initialization

The invariant must be true before the first iteration begins.

2. Maintenance

If the invariant is true before iteration k, it must remain true before iteration k+1.

3. Termination

When the loop finally ends, the invariant (together with the termination condition) provides something useful: a proof that the algorithm is correct.

Insertion Sort

The Algorithm Idea

Think about how we sort playing cards in our hand. We pick up cards one by one.

Each new card is inserted into its correct position among the cards we're already holding.

At any point, the cards in our hand are sorted we're just growing that sorted portion one card at a time.

Pseudocode

INSERTION-SORT(A):

for j = 2 to n:

key = A[j]

i = j - 1

while i > 0 and A[i] > key:

A[i+1] = A[i]

i = i - 1

A[i+1] = key

The Loop Invariant for Insertion Sort

At the start of each iteration of the outer for loop, the subarray A[1..j-1] consists of the elements originally in A[1..j-1], but in sorted order.

Now let's prove it using all three obligations:

Initialization (j = 2):

Before the first iteration, A[1..j-1] = A[1..1], which is a single element.

A single element is trivially sorted.

The invariant holds.

Insertion Sort

Maintenance:

Suppose the invariant holds at the start of iteration **j** — meaning **A[1..j-1]** is sorted. The body of the loop takes **A[j]** (the "key") and shifts elements in **A[1..j-1]** that are larger than the key one position to the right, then drops the key into its correct position. After this, **A[1..j]** is sorted. So at the start of the next iteration **(j+1), A[1..j]** is sorted — the invariant holds for **j+1**.

Termination:

The loop terminates when **j = n + 1.** Substituting into the invariant:

A[1..n] is sorted. That's the entire array, the algorithm is correct.

Complexity Analysis of Insertion Sort

Operation	Cost	Times Executed
`for j = 2 to n`	c_1	n
`key = A[j]`	c_2	n-1
`i = j - 1`	c_3	n-1
`while` condition check	c_4	Σt_j (varies)
`A[i+1] = A[i]`	c_5	$\Sigma(t_j - 1)$
`i = i - 1`	c_6	$\Sigma(t_j - 1)$
`A[i+1] = key`	c_7	n-1

Where $\mathbf{t_j}$ is the number of times the while loop executes for a given **j**.

t_j depends entirely on the input. This is where best, average, and worst cases diverge.

Insertion Sort

Best Case — Already Sorted Array:

If the array is already sorted, then for every j, A[j-1] ≤ A[j], so the while loop never executes its body. This means $t_j = 1$ for all j (only the condition is checked, immediately failing).

Total cost:

$T(n) = c_1n + c_2(n-1) + c_3(n-1) + c_4(n-1) + c_7(n-1)$

This simplifies to $T(n) = an + b$ for some constants a and b.

Best Case: $\Theta(n)$ linear time.

Worst Case (Reverse Sorted Array):

If the array is sorted in reverse, then every new element must be compared against all previously sorted elements. So $t_j = j$ for each iteration.

This means:

$\Sigma t_j = 2 + 3 + 4 + ... + n = n(n+1)/2 - 1$

$\Sigma(t_j - 1) = 1 + 2 + ... + (n-1) = n(n-1)/2$

Total cost becomes a quadratic function:

$T(n) = an^2 + bn + c$

Worst Case: $\Theta(n^2)$Insertion Sort

Average Case:

On average, each element A[j] needs to be compared to about half of the already-sorted elements. So on average $t_j \approx j/2$.

Summing these still gives a quadratic: $\Theta(n^2)$. average case doesn't always rescue. For Insertion Sort, it doesn't.

When is Insertion Sort actually good?

Despite its $\Theta(n^2)$ worst case, Insertion Sort is excellent for:

Small arrays ($n \leq 20$ or so) constants matter at small sizes Nearly sorted data it approaches $\Theta(n)$ in the best case Online algorithms it can sort elements as they arrive Many production sorting algorithms (like Timsort, used in Python and Java) use Insertion Sort internally for small subarrays.

Selection Sort

The Idea

Find the minimum element in the unsorted portion, swap it to its correct position, then shrink the unsorted portion by one.

SELECTION-SORT(A):

for i = 1 to n-1:

min_idx = i

for j = i+1 to n:

if A[j] < A[min_idx]:

min_idx = j

swap A[i] and A[min_idx]

Selection Sort

Loop Invariant for Selection Sort

At the start of each iteration of the outer loop, A[1..i-1] contains the i-1 smallest elements of the original array, in sorted order.

Initialization:

Before the first iteration, i=1, so A[1..0] is empty, trivially satisfies the condition.Selection Sort

Maintenance:

The inner loop finds the minimum of A[i..n]. After the swap, A[i] holds the i-th smallest element. So A[1..i] now contains the i smallest elements in sorted order the invariant is maintained for i+1.

Termination:

When i = n, A[1..n-1] contains the n-1 smallest elements in sorted order, which forces A[n] to be the largest. The whole array is sorted.

Complexity Analysis

The inner loop runs (n-i) times for each value of i:

Total comparisons = (n-1) + (n-2) + ... + 1 = n(n-1)/2

This is always $\Theta(n^2)$ regardless of input. Unlike Insertion Sort, Selection Sort has no best case benefit. Even if the array is already sorted, it still does all the comparisons.

However, Selection Sort does at most n-1 swaps (one per outer iteration). Insertion Sort can do up to $\Theta(n^2)$ shifts. So if writing to memory is very expensive, Selection Sort can be preferable.

Bubble Sort

Repeatedly walk through the array, comparing adjacent pairs and swapping them if they're out of order. Each full pass bubbles the largest unsorted element to its correct position at the end.

BUBBLE-SORT(A):

for i = 1 to n-1:

for j = 1 to n-i:

if A[j] > A[j+1]:

swap A[j] and A[j+1]

Bubble Sort

Loop Invariant

After the i-th pass of the outer loop, the last i elements A[n-i+1..n] are in their final sorted positions.

Complexity

Worst Case (reverse sorted): $\Theta(n^2)$ comparisons and swaps

Best Case (already sorted, with optimization): If a flag is added to detect whether any swap occurred in a pass, and break early if none

did → $\Theta(n)$

Average Case: $\Theta(n^2)$

Stability

If two elements are equal, do they stay in their original relative order?

Insertion and Bubble sort are stable because they only swap when strictly necessary.

Selection Sort is not stable because its long-range swaps can disrupt the relative order of equal elements.

The Loop Invariant as a Design Tool

Loop invariants aren't just for verifying algorithms after the fact they're a design tool.

While designing an algorithm, ask the question: what property do I want to be true at the end of each iteration? If that can be stated clearly, then it is essentially

described what the loop needs to do. The algorithm then follows from the invariant.

For example, if the invariant is A[1..j] is sorted, then the loop body must ensure that adding element A[j+1] preserves that sorted order, and that's exactly what Insertion Sort's inner while loop does. This way of thinking define the invariant, then write code to maintain it is a fundamental algorithm design skill that you'll use throughout this course (in Divide and Conquer, Dynamic Programming, Graph algorithms, etc.).

CHP 4: Sorting Algorithm Analysis

The Sorting Problem

Before analyzing any algorithm, We must state the problem precisely.

Input: A sequence of n numbers $\langle a_1, a_2, ..., a_n \rangle$

Output: A permutation (reordering) $\langle a'_1, a'_2, ..., a'_n \rangle$ of the input sequence

such that:

$$a'_1 \leq a'_2 \leq a'_3 \leq \cdots \leq a'_n$$

The numbers we wish to sort are also known as **keys**. In practice, each key usually comes attached to **satellite data** — other information associated with that **key**. For example, sorting a list of students by GPA: the GPA is the key, but each student has a name, ID, course history, etc. The sorting algorithm moves keys around, and satellite data travels with them.

The Computational Model

RAM (Random Access Machine) model:

• Instructions execute one at a time, sequentially

• Each basic operation (arithmetic, comparison, assignment, array access, control flow) takes constant time

• Memory access takes constant time regardless of location

• We do not model cache effects, pipelining, or parallelism This is an abstraction, but a useful and standard one. It lets us count operations cleanly.

What We Count

We measure running time as a function of input size n. The running time is the number of primitive operations executed, where each operation costs a constant amount $\mathbf{c_i}$.

Rather than adding up exact constants (which depend on the machine and implementation), we care about how the total count grows as n grows — which is where asymptotic notation comes in.

Cases of Analysis

Because the running time may depend not just on the size of the input but on its specific values, we analyze:

Worst case:

The input that causes the maximum number of operations.

Best case:

The input that causes the minimum number of operations.

Average case:

Expected running time over all inputs of size n.

Insertion Sort

INSERTION-SORT(A)

```
1 for j = 2 to n
2 key = A[j]
3 // Insert A[j] into the sorted sequence A[1..j-1]
4 i = j - 1
5 while i > 0 and A[i] > key
6 A[i+1] = A[i]
7 i = i - 1
8 A[i+1] = key
```

Insertion Sort

• The outer for loop iterates j from 2 to n. At the start of each iteration, A[1..j-1] is already sorted.

• key = A[j] saves the current element we want to insert.

• The inner while loop walks backwards through the sorted portion, shifting elements one position to the right as long as they are greater than key

• When the while loop ends, A[i+1] = key drops key into its correct position.

• After this, A[1..j] is sorted, setting up the next iteration correctly.

Proving Correctness

Loop Invariant: At the start of each iteration of the for loop (line 1), the subarray A[1..j-1] consists of the elements originally in A[1..j-1] but in sorted order.

Two things are being claimed simultaneously:

• A[1..j-1] is sorted (in nondecreasing order)

• A[1..j-1] contains exactly the original elements that were in those positions (no elements have been lost or duplicated — they've just been rearranged)

Analyzing Running Time

The Cost Table

Line	Code	Cost	Times Executed
1	`for j = 2 to n`	c_1	n
2	`key = A[j]`	c_2	$n-1$
4	`i = j - 1`	c_4	$n-1$
5	`while i > 0 and A[i] > key`	c_5	$\sum_{j=2}^{n} t_j$
6	`A[i+1] = A[i]`	c_6	$\sum_{j=2}^{n}(t_j - 1)$
7	`i = i - 1`	c_7	$\sum_{j=2}^{n}(t_j - 1)$
8	`A[i+1] = key`	c_8	$n-1$

Analyzing Running Time

Line 1 executes n times: the for loop header is evaluated once for each value j ∈ {2, 3, ..., n} plus one final time when j = n+1 to discover the loop should terminate. That's (n−1) + 1 = n times. t_j is defined as the number of times the while loop condition on line 5 is tested for a given value of j. This includes the final test that fails (causing the while loop to exit). So if the while body executes k times for a given j, then $t_j = k + 1$. Line 6 and 7 (the while loop body) execute $t_j - 1$ times for each j, because they run one fewer time than the condition is tested (the last test causes an exit without executing the body).

The Total Running Time Formula

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^{n} t_j + c_6 \sum_{j=2}^{n} (t_j - 1) + c_7 \sum_{j=2}^{n} (t_j - 1) + c_8(n-1)$$

This is exact. Now we need to evaluate, $\sum_{j=2}^{n} t_j$ and this depends entirely on the input. This is where cases diverge.

Best Case Analysis

What input produces the best case?

An array that is already sorted in nondecreasing order.

Why? When A is already sorted, for every j, A[j-1] ≤ A[j] ≤ A[j]. So when we set key = A[j] and check the while condition: A[i] > key is

immediately false (since A[i] = A[j-1] ≤ A[j] = key). The while loop body never executes.

Therefore: $t_j = 1$ for each j (the condition is checked exactly once and fails).

$$\sum_{j=2}^{n} t_j = \sum_{j=2}^{n} 1 = n - 1$$

$$\sum_{j=2}^{n} (t_j - 1) = \sum_{j=2}^{n} 0 = 0$$

Best Case Analysis

Substituting

$$T\ n = c1n + c2(\ n - 1) + c4\ (n - 1\ ) + c5\ (n - 1\ ) + c6 \cdot 0 + c7 \cdot 0 + c8\ (n - 1\ )$$

$$T\ n = c1n + (c2 + c4 + c5 + c8\ )n - 1$$

$$T\ n = (c1 + c2 + c4 + c5 + c8)\ n - (c2 + c4 + c5 + c8)$$

This is of the form:

$$T\ n = an + b$$

$$TBest\ n$$

$$= \Theta(n)$$

This is linear time. For every additional element you add to an already sorted array, the cost grows by a fixed constant amount.

Worst Case Analysis

What input produces the worst case?

An array sorted in strictly decreasing (reverse) order.

Why? When the array is reverse sorted, every element A[j] is smaller than all elements before it. So when we pick key = A[j], the while loop must compare against every single element in A[1..j-1] — none of them can be ≤ key — and shift all of them right.

Therefore: $t_j = j$ for each j.

The while condition is checked j times for each j: once for each of the j−1 elements that are larger than key, and one final time when i = 0 (when i > 0 fails).

$$\sum_{j=2}^{n} t_j = \sum_{j=2}^{n} j = (2 + 3 + 4 + \cdots + n)$$

Worst Case Analysis

Using the arithmetic series formula:

$$\sum_{j=2}^{n} j = \frac{n(n+1)}{2} - 1$$

$$\sum_{j=2}^{n}(t_j - 1) = \sum_{j=2}^{n}(j-1) = (1 + 2 + 3 + \cdots + (n-1)) = \frac{n(n-1)}{2}$$

Substituting into the total cost formula:

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) + c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1)$$

Expanding and grouping by powers of n:

$$T(n) = \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right) n - (c_2 + c_4 + c_5 + c_8)$$

Worst Case Analysis

$$T(n) = an^2 + bn + c$$
$$T_{worst}(n) = \Theta(n^2)$$

This is quadratic time. Doubling the input size roughly quadruples the running time.

Average Case Analysis

What input produces the average case?

We assume the input is a random permutation of n distinct numbers — each of the n! permutations equally likely.

Computing the expected value of t_j:

For a given j, consider inserting A[j] into the sorted subarray A[1..j-1].

Since the input is random, A[j] is equally likely to be the smallest, 2nd smallest, ..., or j-th smallest among the j elements {A[1], ..., A[j-1], A[j]}.

Average Case Analysis

If A[j] is the largest among the j elements: the while loop never executes its body. $t_j = 1$.

If A[j] is the 2nd largest: the while loop executes once. $t_j = 2$.

...

If A[j] is the smallest: the while loop executes $j-1$ times. $t_j = j$.

Each case occurs with probability 1/j. Therefore:

$$E[t_j] = \frac{1}{j}(1+2+3+\cdots+j) = \frac{1}{j}\cdot\frac{j(j+1)}{2} = \frac{j+1}{2}$$

Summing:

$$\sum_{j=2}^{n} E[t_j] = \sum_{j=2}^{n} \frac{j+1}{2} = \frac{1}{2}\sum_{j=2}^{n}(j+1) = \frac{1}{2}(3+4+\cdots+(n+1)) = \frac{1}{2}\left(\frac{n(n+1)}{2}+n-1\right) \approx \frac{n^2}{4}$$

This is still quadratic roughly half of the worst case, but same

asymptotic growth.

$Tavg\ n = \Theta(n\ 2)$

The average case does not rescue Insertion Sort. For random inputs, it

is still quadratic.

(5)Recursion, Recurrences, and the Recursion Tree

Recursion:

Some times problems are too difficult or too complex to solve because they are too big. A problem can be broken down into sub-problems and we can find a way to solve these subproblems. Then build up to a solution to the entire problem.

This is the idea behind recursion

Recursive algorithms break down problem in pieces which we either already know the answer to, or can be solved applying same algorithm to each piece And finally combine the results of these sub-problems to find the final solution

More concisely, a recursive function or definition is defined in terms of itself. Recursion is a computer algorithm that calls itself in steps having a termination condition.

What Recursion Actually Is

- When a function calls itself, the computer does not magically know how to handle it. It follows a precise mechanical process using the call stack.
- Every time any function is called (recursive or not) the computer creates a stack frame for that call. A stack frame stores everything that call needs to execute: the function's local variables, its parameters, and a return address

telling the processor where to go when this call finishes. Stack frames are pushed onto the call stack when a function is called and popped off when it returns.

- For a recursive function, each recursive call pushes a new frame. The stack grows deeper with each call. When the base case is reached, frames start popping — each returning its result to the frame below it.

Consider factorial: Computing Factorial(4)produces this call stack:

Factorial(n): Factorial(4) → calls Factorial(3)

if n == 0: Factorial(3) → calls Factorial(2)

return 1 Factorial(2) → calls Factorial(1)

Factorial(1) → calls Factorial(0)

Factorial(0) → returns 1 ← base case

return n * Factorial(n-1) returns 1 * 1 = 1

returns 2 * 1 = 2

returns 3 * 2 = 6

returns 4 * 6 = 24 Each level waits for the level below it to complete before it

can finish. The stack reaches depth n before anything returns. This means Factorial uses O(n) space — one stack frame per level, n levels deep. Every recursive call is not free. It consumes memory proportional to the depth of recursion.

Requirements of Every Correct Recursive Algorithm: A recursive algorithm is correct if and only if it satisfies three conditions.

Condition 1 (Base case): There must be at least one input for which the function returns a result directly without making a recursive call. Without a base case the function calls itself forever, the stack grows without bound,

and the program crashes with a stack overflow.

Condition 2 (Recursive case): For inputs that are not base cases, the function must make one or more recursive calls.

Condition 3 (Progress): Every recursive call must be on a strictly smaller or simpler input that moves toward the base case. This is what guarantees termination.

These three conditions are not style guidelines. They are the minimum requirements for a recursive algorithm to be correct and terminating.

Recurrence Relation: When analyzing the running time of a recursive algorithm, we get an equation that defines running time in terms of itself on smaller inputs. That equation is called a recurrence relation.

A recurrence relation for an algorithm's running time T(n) has two parts:

The recursive case: expresses T(n) in terms of T on smaller inputs. For example $T(n) = 2T(n/2) + n$ says: to solve a problem of size n, we make two recursive calls each on a problem of size n/2, and we do n units of non recursive work.

The base case: gives T directly for small inputs. Usually $T(1) = \Theta(1)$ or $T(0) = \Theta(1)$ meaning solving a problem of size 1 takes constant time.

Solving a recurrence means finding a closed-form expression (something like $\Theta(n \log n)$ or $\Theta(n^2)$) that satisfies the equation. That closed form is the running time of the algorithm.

Translating Code Into Recurrences:

We need to look at a recursive algorithm and systematically extract its recurrence. The process has three **steps:**

Step 1 (Count the recursive calls and their sizes) How many recursive calls are made? What size is each subproblem? If the algorithm calls itself twice on inputs of size n/2, that contributes 2T(n/2) to the recurrence.

Step 2 (Identify the non-recursive work) What work does the function do outside of the recursive calls? If it scans the input once, that is $\Theta(n)$.

If it does a constant amount of work, that is $\Theta(1)$.

Step 3 (Write the recurrence) Combine: T(n) = (number of calls) × T(subproblem size) + (non-recursive work).

BinarySearch(A, lo, hi, x):

```
if lo > hi: return -1          ← Θ(1) work
mid = (lo + hi) / 2            ← Θ(1) work
```

if A[mid] == x: return mid ← Θ(1) work

if A[mid] < x:

return BinarySearch(A, mid+1, hi, x) ← one call, size n/2

else:

return BinarySearch(A, lo, mid-1, x) ← one call, size n/2

One recursive call, subproblem size n/2, non-recursive work Θ(1): $T(n) = T(n/2) + \Theta(1)$, $T(1) = \Theta(1)$

MergeSort(A, lo, hi):

if lo >= hi: return ← Θ(1) work

mid = (lo + hi) / 2

MergeSort(A, lo, mid) ← one call, size n/2

MergeSort(A, mid+1, hi) ← one call, size n/2

Merge(A, lo, mid, hi) ← Θ(n) work

Two recursive calls, each on size n/2, non-recursive work Θ(n) for the merge:

$T(n) = 2T(n/2) + \Theta(n)$, $T(1) = \Theta(1)$

Fibonacci(n):

if n <= 1: return n ← Θ(1)

return Fibonacci(n-1) + Fibonacci(n-2) ← two calls, sizes n-1 and n-2

Two recursive calls of sizes n-1 and n-2, non-recursive work Θ(1):

$T(n) = T(n-1) + T(n-2) + \Theta(1)$, $T(0) = T(1) = \Theta(1)$

Subproblems decrease by 1, not by a factor.

That is the mathematical explanation for why naive Fibonacci is catastrophically slow.

The Three Canonical Forms

Before learning to solve recurrences, recognize the three forms that appear most frequently:

Form 1:

T(n) = T(n/2) + Θ(1) One call, half the size, constant work per level. This is Binary Search.

Solution: Θ(log n). Each level does constant work and there are log n levels.

Form 2:

T(n) = T(n-1) + Θ(1) One call, size decreases by one, constant work. This is linear recursion like factorial.

Solution: Θ(n). There are n levels each doing constant work.

Form 3:

T(n) = 2T(n/2) + Θ(n) Two calls, half the size each, linear work per level. This is Merge Sort.

Solution: Θ(n log n). There are log n levels each doing Θ(n) total work.

These three forms between them cover an enormous fraction of the algorithms we will encounter. When a new recurrence appears, check if it matches or resembles one of these before doing any calculation.

The Recursion Tree Method

The recursion tree method is the most intuitive way to solve a recurrence. It is also the best way to build intuition for why a recurrence has the solution it does, and it is the method that leads most naturally to guesses for the substitution method.

The idea is to draw the recursion as a tree and then sum up all the work done across all nodes.

How to draw and use the recursion tree:

Each node represents a subproblem. The value at each node is the non recursive work done at that subproblem(not including recursive calls). The children of a node are its recursive subproblems. The total cost of the algorithm is the sum of all node values across the entire tree.

Three quantities to compute for each tree:

One (the cost at each level of the tree).

Two (the number of levels).

Three (the sum across all levels).

Example

T(n) = 2T(n/2) + n — Merge Sort

At every level, the total cost is exactly n.

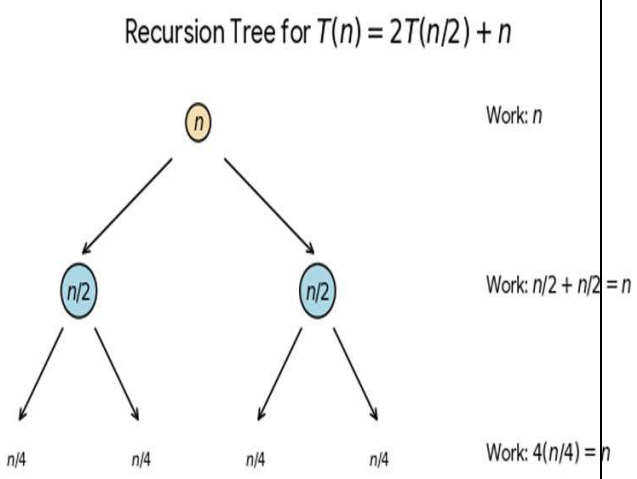
How many levels are there? The subproblem

size at level n is n/2i . The tree stops when the subproblem size hits

1(n/2i = 1). To find i we solve 2i = n which gives us i = log2n

Total cost = n × (log2n) = Θ(n log n).

The even distribution is characteristic of T(n) = 2T(n/2) +n.



Example

T(n) = T(n/2) + 1 — Binary Search

Level 0: 1 → cost: 1

Level 1: 1 → cost: 1

Level 2: 1 → cost: 1

...

Level k: 1 → cost: 1

One node per level, each doing 1 unit of work. Number of levels: log n (subproblem halves each time).

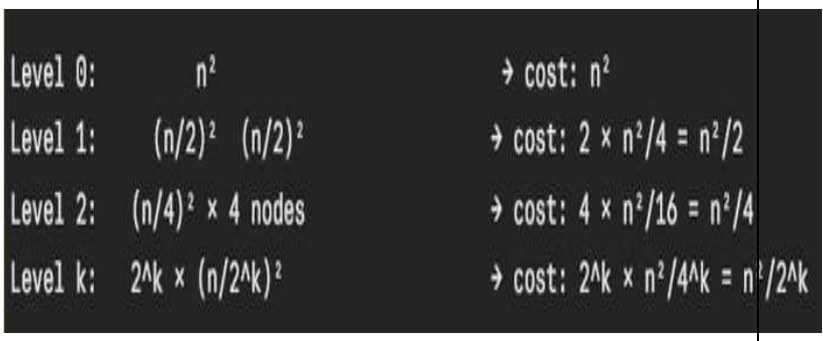
Total cost = 1 × log n = Θ(log n).

Example

T(n) = 2T(n/2) + n^2 — Work dominated by root

The cost per level is n^2, $n^2/2$, $n^2/4$, $n^2/8$, ... — a geometric series with ratio 1/2.

Total cost = $n^2 \times (1 + 1/2 + 1/4 + ...) = n^2 \times 2 = \Theta(n^2)$.



The Three Behaviors

These three examples reveal the three fundamental behaviors of divide-and conquer recurrences:

Leaves dominate: When recursive calls generate more and more work at deeper levels, the leaves of the tree do most of the work. The cost is determined by the number of leaves.

Even distribution: When each level does the same total work, cost is (work per level) × (number of levels).

Root dominates: When the non-recursive work shrinks geometrically as you go deeper, the root does most of the work and lower levels form a convergent series.

This trichotomy is exactly the intuition behind the three cases of the Master Theorem.

The Geometric Series

Summing levels in a recursion tree almost always produces a geometric series.

For a geometric series with ratio r:

If $r < 1$: $1 + r + r^2 + ... = 1/(1-r) \rightarrow$ converges to a constant. The first term dominates.

If $r = 1$: $1 + 1 + 1 + ...$ k times $= k \rightarrow$ linear in the number of terms.

If $r > 1$: $1 + r + r^2 + ... + r^k = (r^{(k+1)} - 1)/(r - 1) \approx r^k \rightarrow$ the last term dominates.

These three cases correspond directly to root-dominated, evenly distributed, and leaf-dominated recursion trees respectively.

The Substitution Method

The substitution method has two steps:

Step 1 Guess a closed-form solution.

Use your recursion tree, use the canonical forms, use intuition.

Step 2 Prove the guess is correct using mathematical induction.

Substitute your guess into the recurrence and verify the inequality holds.

The word substitution refers to Step 2 you substitute your guess into the recurrence relation in place of T and verify it is consistent.

The Structure of the Inductive Proof

The proof always has this structure:

Inductive hypothesis:

Assume $T(k) \leq c \cdot f(k)$ for all $k < n$ (for Big-O proofs)

or $T(k) \geq c \cdot f(k)$ for all $k < n$ (for Big-Ω proofs), where c is a constant you will determine.

Inductive step:

Use this assumption to show $T(n) \leq c \cdot f(n)$ (or $\geq$ for Ω).

Base case:

Verify the bound holds for small n directly.

Prove T(n) = 2T(n/2) + n is O(n log n)

Guess: $T(n) = O(n \log n)$, so we want to show $T(n) \leq c \cdot n \log n$ for some constant $c > 0$.

Inductive hypothesis: Assume $T(n/2) \leq c \cdot (n/2) \cdot \log(n/2)$.

Inductive step: Substitute into the recurrence:

$T(n) = 2T(n/2) + n \leq 2 \cdot c \cdot (n/2) \cdot \log(n/2) + n$

$= c \cdot n \cdot \log(n/2) + n$

$= c \cdot n \cdot (\log n - \log 2) + n$

$= c \cdot n \cdot \log n - c \cdot n \cdot \log 2 + n$

Prove T(n) = 2T(n/2) + n is O(n log n)

$= c \cdot n \cdot \log n - c \cdot n + n \leftarrow$ since $\log 2 = 1$

$= c \cdot n \cdot \log n + n(1 - c)$

For this to be $\leq c \cdot n \cdot \log n$, we need $n(1 - c) \leq 0$, which holds whenever $c \geq 1$.

So for any $c \geq 1$, $T(n) \leq c \cdot n \cdot \log n$.

Base case: We need $T(1) \leq c \cdot 1 \cdot \log 1 = 0$. But $T(1) = \Theta(1) > 0$. Problem.

The base case fails at $n = 1$ because $\log 1 = 0$.

The fix: we do not need the base case to be $n = 1$. We need it to hold for some small n. In this case verify it holds for $n = 2$ and $n = 3$ (the first values where $\log n > 0$), and choose c large enough to cover them. The recurrence is only valid for large n anyway.

Result: $T(n) \leq c \cdot n \cdot \log n$ for all $n \geq 2$ with $c \geq 1$. Therefore $T(n) = O(n \log n)$.

Prove $T(n) = T(n-1) + n$ is $O(n^2)$

Guess: $T(n) = O(n^2)$, want $T(n) \leq c \cdot n^2$.

Inductive hypothesis: $T(n-1) \leq c \cdot (n-1)^2$.

Inductive step:

$T(n) = T(n-1) + n \leq c \cdot (n-1)^2 + n$

$= c \cdot (n^2 - 2n + 1) + n$

$= c \cdot n^2 - 2cn + c + n$

$= c \cdot n^2 + n(1 - 2c) + c$

For this to be $\leq c \cdot n^2$, we need $n(1 - 2c) + c \leq 0$. For $c \geq 1$. Choose $c = 1$, $n_0 = 1$.

Therefore $T(n) = O(n^2)$.

The Strengthened Hypothesis Technique

Sometimes a direct induction attempt fails even when the guess is correct, because the algebra does not quite work out. The fix is to subtract a lower-order term from your hypothesis.

Example: $T(n) = 2T(n/2) + 1$

Guess $T(n) = O(n)$. **Hypothesis:** $T(n/2) \leq c \cdot (n/2)$

$T(n) = 2T(n/2) + 1 \leq 2 \cdot c \cdot (n/2) + 1 = c \cdot n + 1$

This gives $c \cdot n + 1$, not $c \cdot n$. The extra +1 prevents the proof from closing.

Strengthen the hypothesis: assume $T(n/2) \leq c \cdot (n/2) - d$ for some additional constant $d > 0$.

$T(n) \leq 2 \cdot (c \cdot (n/2) - d) + 1 = c \cdot n - 2d + 1 \leq c \cdot n - d$ when $d \geq 1$.

Now it closes. $T(n) \leq c \cdot n - d \leq c \cdot n$.

This technique subtracting a lower-order term from the hypothesis appears constantly in substitution method proofs and is essential to know.

The Master Theorem: The substitution method works for any recurrence you can guess correctly, but it requires a separate proof for each case. The Master Theorem

packages an entire family of proofs into one reusable result.

The form it applies to:

$T(n) = aT(n/b) + f(n)$

where $a \geq 1$, $b > 1$ are constants and $f(n)$ is an asymptotically positive function.

What a, b, and f(n) represent:

a = number of recursive subproblems. For Merge Sort, a = 2.

b = factor by which the problem size shrinks. For Merge Sort, b = 2. Each subproblem is half the size.

f(n) = cost of the non-recursive work at each level the divide and combine steps.

For Merge Sort, $f(n) = \Theta(n)$.

The critical quantity: n^(log_b(a))

This number n to the power of log base b of a appears throughout the Master Theorem. It represents the number of leaves in the recursion tree. Understanding where it comes from makes the theorem feel inevitable rather than mysterious.

At depth k in the recursion tree there are a^k subproblems each of size n/b^k. The leaves are reached when $n/b^k = 1$, so $k = \log_b(n)$. The number of leaves

is therefore $a^{(\log_b(n))} = n^{(\log_b(a))}$. This is just a logarithm identity: $a^{(\log_b(n))} = n^{(\log_b(a))}$.

So $n^{(\log_b(a))}$ is the leaf count. The three cases of the Master Theorem are about whether the non-recursive work f(n) is smaller than, equal to, or larger than the leaf count.

The Three Cases:

Case 1 Leaves dominate:

If $f(n) = O(n^{(\log_b(a) - \varepsilon)})$ for some constant $\varepsilon > 0$, meaning f(n) grows strictly slower than $n^{(\log_b(a))}$, then:

$T(n) = \Theta(n^{(\log_b(a))})$

The leaves of the recursion tree do most of the work. The non-recursive work per level is dwarfed by the leaf count. Solution is determined entirely by the number of leaves.

Case 2 — Even distribution:

If $f(n) = \Theta(n^{(\log_b(a))})$, meaning f(n) grows at the same rate as $n^{(\log_b(a))}$, then:

$T(n) = \Theta(n^{(\log_b(a))} \cdot \log n)$

The work is spread evenly across all levels. Each level contributes the same total, and there are log n levels, so multiply by log n.

Case 3 Root dominates:

If $f(n) = \Omega(n^{(\log_b(a) + \varepsilon)})$ for some $\varepsilon > 0$, meaning f(n) grows strictly faster than $n^{(\log_b(a))}$, AND the regularity condition holds ($af(n/b) \leq cf(n)$ for some $c < 1$ and large n), then:

$T(n) = \Theta(f(n))$

The root does most of the work. The non-recursive cost dominates the leaf cost, and deeper levels contribute geometrically less. The regularity condition ensures the work does not somehow grow going down.

Applying the Master Theorem

Example 1: Merge Sort — $T(n) = 2T(n/2) + n$

$a = 2, b = 2, f(n) = n.$

$n^{(\log_b(a))} = n^{(\log_2(2))} = n^1 = n.$

Compare $f(n) = n$ with $n^{(\log_b(a))} = n$. They are equal. → Case 2.

$T(n) = \Theta(n \log n).$

Example 2: Binary Search — $T(n) = T(n/2) + 1$

a = 1, b = 2, f(n) = 1.

n^($\log_2(1)$) = n^0 = 1.

f(n) = 1 = Θ(1) = Θ(n^(log_b(a))). → Case 2.

T(n) = Θ(log n).

Example 3: T(n) = 4T(n/2) + n

a = 4, b = 2, f(n) = n.

n^($\log_2(4)$) = n^2.

f(n) = n = O(n^(2-1)) = O(n^(log_b(a) - ε)) with ε = 1. → Case 1.

T(n) = Θ(n^2). Leaves dominate.

Example 4: T(n) = 4T(n/2) + n^2

a = 4, b = 2, f(n) = n^2.

n^($\log_2(4)$) = n^2.

f(n) = n^2 = Θ(n^(log_b(a))). → Case 2.

T(n) = Θ(n^2 log n).

Example 5: T(n) = 4T(n/2) + n^3

a = 4, b = 2, f(n) = n^3.

n^($\log_2(4)$) = n^2.

f(n) = n^3 = Ω(n^(2+1)) with ε = 1. Check regularity: a f(n/b) = 4·$(n/2)^3$ = $4n^3/8 = n^3/2$ ≤ (1/2)·f(n). c = 1/2 < 1. → Case 3.

T(n) = Θ(n^3). Root dominates.

When the Master Theorem Does NOT Apply

The Master Theorem only handles recurrences of the form T(n) = aT(n/b) + f(n) with constant a and b. It fails in these situations:

Non-constant number of subproblems:

T(n) = nT(n/2) + f(n). Here a = n is not a constant it grows with n. Master Theorem cannot be applied.

Subtractive recurrences:

T(n) = T(n-1) + f(n). The subproblem size decreases by subtraction, not division. These must be solved by telescoping or substitution.

Unequal subproblem sizes:

T(n) = T(n/3)+ T(2n/3) + n. Two subproblems of different sizes. Not the form aT(n/b). Solve using the recursion tree

method.

The gap between cases:

If f(n) = n^(log_b(a))·log n, this is not Θ(n^(log_b(a))) nor Ω(n^(log_b(a) + ε)). It falls in a gap that Case 2 does not cover in its basic form. CLRS notes this gap and discusses extended versions. For this course, recognize the gap exists and fall back to substitution or the recursion tree.

(6)The Divide-and-Conquer Paradigm:

The Paradigm:

Every divide-and-conquer algorithm has exactly three phases:

Divide: Split the problem into subproblems that are smaller instances of the same problem.

The subproblems do not have to be equal in size, but they must be strictly smaller.

Conquer: Solve each subproblem recursively.

If a subproblem is small enough (below some threshold) solve it directly without further recursion.

This direct solution is the base case.

Combine: Merge the solutions to the subproblems into a solution for the original problem.

The sum of these two steps(Divide + Combine) is exactly the f(n) term in the recurrence.

The key question for any divide-and-conquer algorithm is: what is the total cost of work outside the recursive calls (Divide + Combine)?

This total cost determines whether the algorithm is efficient.

If this combined cost is cheap (O(1) or O(log n)), the algorithm is typically very fast.

If it is expensive ($O(n^2)$), you might not gain much over a direct approach.

Merge Sort:

Merge Sort is divide-and-conquer in its purest form.

It is the template for dozens of other algorithms.

The algorithm:

MergeSort(A, lo, hi):

if lo >= hi: ←base case: one element, already sorted

return

mid = ⌊(lo + hi) / 2⌋

MergeSort(A, lo, mid) ←conquer left half

MergeSort(A, mid+1, hi) ←conquer right half

Merge(A, lo, mid, hi) ←combine: merge two sorted halves

The Merge operation: The Merge step takes two sorted subarrays A[lo..mid] and A[mid+1..hi] and merges them into a single sorted subarray.

It does this by maintaining two pointers (one into each half) and repeatedly choosing the smaller of the two current elements.

Merge Sort

Merge(A, lo, mid, hi):

create temporary arrays L and R

L = A[lo..mid], R = A[mid+1..hi]

i = 0, j = 0, k = lo

while i < len(L) and j < len(R):

if L[i] <= R[j]:

A[k] = L[i], i++

else:

A[k] = R[j], j++

k++

copy remaining elements of L or R into A.

Merge scans each element of both halves exactly once.

Total work: Θ(n). This is the combine cost.

Setting up the recurrence:

Divide: Θ(1) just compute mid.

Conquer: 2 recursive calls, each on n/2 elements.

Combine: Θ(n) for Merge.

T(n) = 2T(n/2) + Θ(n)

From the Master Theorem (Case 2) and the recursion tree that this solves to Θ(n log n).

Merge Sort is optimal because it can be proven (through an

information-theoretic argument) that any comparison-based sorting algorithm must make Ω(n log n) comparisons in the worst case. Merge Sort achieves Θ(n log n).

Therefore Merge Sort is asymptotically optimal among comparison based sorts.

Space cost:

Merge Sort requires O(n) auxiliary space for the temporary arrays used in merging. This is its one disadvantage relative to in-place sorts like Heapsort.

Binary Search:

Binary Search is a degenerate divide-and-conquer. It divides, conquers one subproblem, and combines trivially.

BinarySearch(A, lo, hi, x):

if lo > hi: return-1

mid = ⌊(lo + hi) / 2⌋

if A[mid] == x: return mid

if A[mid] < x: return BinarySearch(A, mid+1, hi, x)

else: return BinarySearch(A, lo, mid-1, x)

Divide: compare x with A[mid] — Θ(1).

Conquer: one recursive call on half the array.

Combine: nothing, the recursive result is returned directly.

$T(n) = T(n/2) + \Theta(1) \rightarrow \Theta(\log n)$

Binary Search illustrates that combine cost of Θ(1) produces logarithmic complexity.

The work is entirely in reducing the problem size, once it is small enough, the answer comes immediately.

Binary Search

Correctness through loop invariant:

The invariant for Binary Search:

at each recursive call, if x is in the array, it is in A[lo..hi].

This is trivially true initially (the whole array), maintained by the halving

(we only discard the half where x cannot be), and at termination either we found x or the search space is empty.

The Maximum Subarray Problem

This problem demonstrates the paradigm on a problem that is not immediately recognizable as recursive.

The problem: Given an array A of n integers (possibly negative), find the contiguous subarray whose sum is maximum.

For example, in A = [-2, 1,-3, 4,-1, 2, 1,-5, 4], the maximum subarray is [4,-1, 2, 1] with sum 6.

Why brute force is slow:

Checking all $O(n^2)$ subarrays and computing their sums takes $O(n^3)$ naively, or $O(n^2)$ with prefix sums. Can divide-and-conquer do better?

The divide-and-conquer insight:

Split the array at its midpoint. The maximum subarray must be in one of three places:

I. Entirely in the left half A[lo..mid]

II. Entirely in the right half A[mid+1..hi]

III. Crossing the midpoint starting somewhere in the left half and ending somewhere in the right half

The first two cases are solved recursively.

The crossing case is handled directly.

Finding the maximum crossing subarray:

The maximum crossing subarray must include A[mid] and A[mid+1].

To find it: scan left from mid, accumulating the maximum left sum

scan right from mid+1, accumulating the maximum right sum.

The crossing maximum is their combination.

This scan takes $\Theta(n)$.

The Maximum Subarray Problem

Find Max Crossing (A, lo, mid, hi):

LeftSum = -∞, sum = 0

for i = mid downto lo:

sum += A[i]

if sum > leftSum: leftSum = sum, maxLeft = i

rightSum = -∞, sum = 0

for j = mid+1 to hi:

sum += A[j]

if sum > rightSum: rightSum = sum, maxRight = j

return (maxLeft, maxRight, leftSum + rightSum)

The full algorithm:

MaxSubarray(A, lo, hi):

if lo == hi: return (lo, hi, A[lo]) ← base case

mid = ⌊(lo + hi) / 2⌋

(lLo, lHi, lSum) = MaxSubarray(A, lo, mid)

(rLo, rHi, rSum) = MaxSubarray(A, mid+1, hi)

(cLo, cHi, cSum) = FindMaxCrossing(A, lo, mid, hi)

return whichever of the three has maximum sum

Recurrence:

Tworecursive calls on n/2, combine step $\Theta(n)$:

$T(n) = 2T(n/2) + \Theta(n) \rightarrow \Theta(n \log n)$

This is better than $O(n^2)$ brute force.

(7)Strassen, Edge Cases, and Synthesis

Strassen's Matrix Multiplication:

The naive algorithm:

Multiplying two n×n matrices using the definition takes $O(n^3)$ time.

For each of the n^2 entries in the result, you compute a dot product of two vectors of length n that is n multiplications and n-1 additions per entry.

Total: $\Theta(n^3)$.

The divide-and-conquer attempt:

A natural divide-and-conquer: split each n×n matrix into four n/2 × n/2

submatrices:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

Then C=A×B can be computed as:

$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21}$

$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22}$

$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21}$

$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22}$

This requires 8 multiplications of n/2 × n/2 matrices and 4 additions. The additions take $\Theta(n^2)$ each since each matrix has $(n/2)^2 = n^2/4$ entries.

Strassen's Matrix Multiplication

Recurrence: $T(n) = 8T(n/2) + \Theta(n^2)$

$n(\log_2 8) = n^3$. $f(n) = n^2 = O(n3\text{-}1)$.

Case 1. $T(n) = \Theta(n^3)$.

No improvement. Eight subproblems of half the size gives back the same cubic complexity.

Strassen's insight:

Volker Strassen discovered in 1969 that you can compute the four C submatrices using only 7 multiplications instead of 8, by defining 7 intermediate matrices cleverly:

$M_1=(A_{11} + A_{22})(B_{11} + B_{22})$

$M_2=(A_{21} + A_{22})B_{11}$

$M_3=A_{11}(B_{12}- B_{22})$

$M_4=A_{22}(B_{21}- B_{11})$

$M_5=(A_{11} + A_{12})B_{22}$

$M_6=(A_{21}- A_{11})(B_{11} + B_{12})$

$M_7=(A_{12}- A_{22})(B_{21} + B_{22})$

Then:

$C_{11} = M_1 + M_4 - M_5 + M_7$

$C_{12} = M_3 + M_5$

$C_{21} = M_2 + M_4$

$C_{22} = M_1 - M_2 + M_3 + M_6$

Seven multiplications of $n/2 \times n/2$ matrices, and $\Theta(n^2)$ additions.

Recurrence:

$T(n) = 7T(n/2) + \Theta(n^2)$

$n\log_2 7 = n2.807$. $f(n) = n^2 = O(n2.807 - \varepsilon)$ with $\varepsilon \approx 0.807$. → Case 1.

$T(n) = \Theta(n\log_2 7) \approx \Theta(n2.807)$

Strassen's Matrix Multiplication

Why this matters:

One fewer recursive call (from 8 to 7) changes the exponent from 3 to 2.807. That seems small. For $n = 1000$, $n2.807 \approx 175{,}000{,}000$ while $n^3 = 1{,}000{,}000{,}000$. Strassen is roughly 5.7 times faster for 1000×1000 matrices. More importantly, Strassen proved that the naive algorithm is

not optimal the exponent can be reduced. This launched a line of research that continues today.

The current best known exponent is approximately 2.371, though those algorithms are only faster than Strassen for astronomically large matrices and are not used in practice.

<u>The Unequal Subproblem Case:</u>

Consider the recurrence: $T(n) = T(n/3) + T(2n/3) + n$

This arises in certain partitioning algorithms where the split is not balanced one subproblem gets a third of the input and the other gets two thirds. The Master Theorem does not apply because the two subproblems have different sizes.

Use the recursion tree:

At level 0: one problem of size n, cost n.

At level 1: problems of size n/3 and 2n/3, cost $n/3 + 2n/3 = n$.

At level 2: problems of size n/9, n/9 (from the n/3 branch) and n/9, 4n/9 (from the 2n/3 branch).

Total cost: $n/9 + 2n/9 + 2n/9 + 4n/9 = n$.

The cost at every level is exactly n. How many levels?

The left branch shrinks by factor 1/3 each level: reaches 1 after $\log_3 n$ levels. The right branch shrinks by factor 2/3 each level: reaches 1 after log3/2n levels.

The tree is not balanced the right branch goes deeper. The deepest path has length log3/2n.

Total cost = n × log3/2n = n × (log n / log(3/2)) = Θ(n log n).

Even with an unequal split, as long as every level does Θ(n) total work and the number of levels is O(log n), the result is Θ(n log n).

This is why Quicksort with a constant-fraction split (even a bad one like 1/10 and 9/10) still gives Θ(n log n) average case.

The split ratio only affects the constant hidden in Θ, not the asymptotic class.

Changing Variables

Some recurrences look non-standard but can be transformed into a form the Master Theorem handles.

Example: T(n) = 2T(√n) + log n

The subproblem size is √n — not n/b for any constant b. Master Theorem does not directly apply.

Substitution: Let m = log n, so n = 2m.

T(2m) = 2T(2m/2) + m

Define S(m) = T(2m). Then:

S(m) = 2S(m/2) + m

This is exactly the Merge Sort recurrence. By Master Theorem Case 2:

S(m) = Θ(mlog m)

Substituting back: m = log n:

T(n) = Θ(log n · log log n)

Full Synthesis

When you encounter a recursive algorithm, here is the complete systematic approach:

Step 1 Write the algorithm clearly Identify the base case, recursive calls, and non recursive work.

Step 2 Set up the recurrence Count recursive calls (a), subproblem size (n/b or n-k), and non-recursive work f(n).

Step 3 Try the Master Theorem first Check if the recurrence is of the form aT(n/b) + f(n) with constant a and b. If so, compute nlogba, compare with f(n), identify the case, state the solution.

Step 4 If Master Theorem fails, draw the recursion tree Compute cost per level, number of levels, sum across levels. This gives you a strong guess.

Step 5 Verify with substitution if rigor is required Take your guess from the recursion tree and prove it by induction.

Complete Analysis of Merge Sort

Algorithm: As stated earlier.

Recurrence: $T(n) = 2T(n/2) + \Theta(n)$, $T(1) = \Theta(1)$.

Master Theorem: a=2, b=2, f(n)=n. $n\log_2 2 = n$. $f(n) = \Theta(n) = \Theta(n\log ba)$.

Case 2. $T(n) = \Theta(n \log n)$.

Recursion tree verification: log n + 1 levels, each costing $\Theta(n)$. Total $\Theta(n \log n)$.

Substitution verification: $T(n) \leq c \cdot n \cdot \log n$ for $c \geq 1$, $n \geq 2$.

Space complexity: O(n) auxiliary for temporary merge arrays. Recursion depth O(log n) for the call stack. Dominant cost is the auxiliary arrays: O(n) space.

All three methods agree. $T(n) = \Theta(n \log n)$ time, O(n) space.

(8)Heaps & Heapsort

Heap:

A heap is a nearly complete binary tree stored implicitly in an array. Nearly complete means all levels are fully filled except possibly the last, which is filled from left to right. This guarantees the height of an n-element heap is exactly ⌊log n⌋.

Stored implicitly in an array means we do not use pointers. The tree structure is encoded by index arithmetic:

For a node at index i (1-indexed):

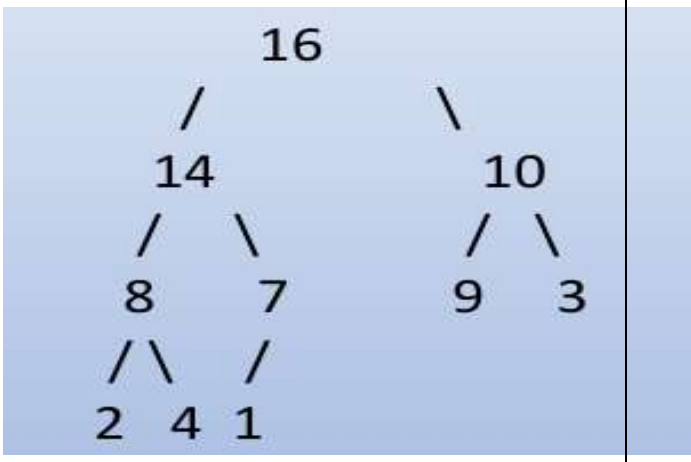
Parent: ⌊i/2⌋

Left child: 2i

Right child: 2i + 1

This is elegant and cache-friendly, the entire tree is a contiguous block of memory with no pointer overhead.

Example: The array [16, 14, 10, 8, 7, 9, 3, 2, 4, 1] represents.



Node at index 1 (value 16) has children at indices 2 (14) and 3 (10). Node at index 4 (value 8) has children at indices 8 (2) and 9 (4). Parent of index 5 (value 7) is index

⌊5/2⌋ = 2 (value 14). The array representation and the tree representation are completely interchangeable.

The Max-Heap Property

A max-heap satisfies: for every node i other than the root, A[Parent(i)] ≥ A[i].

Every node is greater than or equal to both its children. This means the maximum element is always at the root A[1]. You can read the maximum in O(1).

Symmetrically, a min-heap satisfies A[Parent(i)] ≤ A[i] every node is smaller than or equal to its children, minimum is at root. Min-heaps are used for priority queues where you want the minimum. Heapsort uses a max-heap.

The heap property says nothing about the relationship between siblings or between nodes at the same level that are not parent-child.

A heap is not a sorted array it is a partially ordered structure.

Max-Heapify

Max-Heapify is the fundamental heap operation.

It assumes that the binary trees rooted at Left(i) and Right(i) are both valid max-heaps, but that A[i] might be smaller than its children violating the heap property at node i.

Max-Heapify fixes this violation by letting A[i] "float down" to its correct position.

MaxHeapify(A, i, n):

l = 2i, r = 2i + 1

largest = i

if l ≤ n and A[l] > A[largest]: largest = l

if r ≤ n and A[r] > A[largest]: largest = r

if largest ≠ i:

swap A[i] and A[largest]

MaxHeapify(A, largest, n) ← recurse on the subtree we just pushed into

What it does:

Find the largest among A[i], its left child, and its right child. If A[i] is already the largest, the heap property holds and we stop. Otherwise swap A[i] with the largest child and recurse on that child's subtree because pushing A[i] down might have violated the property there.

Time complexity of MaxHeapify:

MaxHeapify makes at most one recursive call, moving one level down the tree each time. The tree has height ⌊log n⌋, so MaxHeapify runs in O(log n) time it processes at most one node per level.

BuildMaxHeap

Given an arbitrary array, BuildMaxHeap converts it into a valid max heap in-place.

BuildMaxHeap(A, n):

for i = ⌊n/2⌋ downto 1:

MaxHeapify(A, i, n)

Why start from ⌊n/2⌋?

Nodes at indices ⌊n/2⌋ + 1 through n are leaves they have no children and trivially satisfy the heap property. Only internal nodes need heapifying. The last internal node is at index ⌊n/2⌋.

Why go downto1(bottom-up)?

MaxHeapify requires that the subtrees below i are already valid heaps. By processing nodes from the bottom up, we ensure this invariant holds when we reach each node the nodes below have already been heapified.

Time complexity the careful analysis:

A naive argument: we call MaxHeapify O(n) times, each taking O(log n). Total O(n log n). Correct but loose.

The tight analysis observes that nodes near the bottom of the tree have smaller subtrees and therefore MaxHeapify on them takes less time.

Specifically, a node at height h takes O(h) time for MaxHeapify (not O(log n) it can only float down h levels from height h).

BuildMaxHeap

In an n-element heap, there are at most ⌈n/2h+1⌉ nodes at height h.

Total cost:

$\sum_{h=0}^{\lfloor \log n \rfloor} \lceil n/2h+1 \rceil \cdot O(h) = O(n \cdot \sum_{h=0}^{\infty} h/2h)$

The sum $\sum h/2h = 2$. Therefore:

BuildMaxHeap runs in Θ(n) time.

This is a crucial and counterintuitive result. Building a heap from an unsorted array is linear not O(n log n). The savings come from the fact that most nodes are near the bottom of the tree and require very little work.

Heapsort With BuildMaxHeap and MaxHeapify in hand, Heapsort is almost trivial:

Heapsort(A, n):

BuildMaxHeap(A, n) ←establish heap property: Θ(n)

for i = n downto 2:

swap A[1] and A[i] ←movemaxtoits final position

MaxHeapify(A, 1, i-1) ←restore heap on reduced array

The idea:

After BuildMaxHeap, A[1] is the maximum element. Swap it with A[n] it is now in its correct sorted position at the end. Reduce the heap size by 1 (exclude A[n] from further consideration) and call MaxHeapify to restore the heap property.

Repeat: A[1] is now the second largest, swap with A[n-1], and so on.

Correctness: At the start of each iteration, A[i+1..n] contains the i largest elements in sorted order, and A[1..i] is a valid max-heap. This is the loop invariant. After n-1 iterations, A[1..n] is fully sorted.

Time complexity:

BuildMaxHeap: Θ(n). The loop runs n-1 times.

Each iteration: one swap O(1) + MaxHeapify O(log i) ≤ O(log n).

Total loop cost: O(n log n).

Overall: Θ(n log n) worst case.

Space complexity: O(1) auxiliary.

This is Heapsort's key advantage over Merge Sort.

Why Heapsort is not used as often as Quicksort in practice:

Despite its optimal worst-case guarantee and in-place operation,

Heapsort has poor cache performance. The access pattern (jumping between parent and child indices that may be far apart in memory) causes frequent cache misses.

Quicksort, despite its $O(n^2)$ worst case, has excellent cache locality and tends to be faster in practice by a constant factor of 2-5×.

The Heap as a Priority Queue

Beyond sorting, heaps implement priority queues data structures supporting:

Maximum: Return the maximum element. A[1] O(1).

Extract-Max: Remove and return the maximum element.

ExtractMax(A, n):

max=A[1]

A[1] = A[n]

n =n-1

MaxHeapify(A, 1, n)

return max

Swap root with last element, reduce size, re-heapify. O(log n).

Increase-Key: Increase the value of an element and restore heap property by floating it up.

IncreaseKey(A, i, key):

A[i] = key

while i > 1 and A[Parent(i)] < A[i]:

swap A[i] and A[Parent(i)]

i = Parent(i)

O(log n) floats up at most log n levels.

The Heap as a Priority Queue

Insert: Add a new element.

Insert(A, key, n):

n =n+1

A[n] =-∞

IncreaseKey(A, n, key)

Insert as-∞ at the end, then increase to the desired key. O(log n).

These four operations

Maximum O(1) Extract-Max O(log n)

Increase-Key O(log n) Insert O(log n)

make heaps the standard priority queue implementation.

They appear as subroutines in Dijkstra's shortest path algorithm, Prim's minimum spanning tree algorithm, and many other graph algorithms.

(9)Hash Tables, Hash Functions, and Chaining

Direct-Address Tables

Suppose every key is a unique integer in the range [0, U-1] for some universe size U.

The simplest possible data structure: an array T of size U, where T[k] stores the element with key k (or NIL if absent).

Search, insert, delete are all O(1) just index directly into the array.

The problem: U can be enormous. If keys are 64-bit integers, $U = 264 \approx 1.8 \times 1019$. Allocating an array of that size is impossible. If keys are strings of length up to 100, U is astronomically larger.

We need a way to use a table of size m (proportional to the number of elements n we actually store, not the universe size U) while preserving near O(1) operations. That is exactly what hashing achieves.

Hashing

Hashing is a technique used to uniquely identify objects and store them efficiently for fast retrieval.

The primary objective of hashing is to implement a Dictionary data structure.

A dictionary supports three basic operations efficiently:

i. Insert a new element

ii. Search for an existing element.

iii. Delete an unwanted element.

Hash Tables & Hash Functions

A hash table is an array T of size m, typically much smaller than U.

A hash function h maps keys from the universe U to slots in T:

$h : U \rightarrow \{0, 1, ..., m-1\}$

To store an element with key k, we store it at slot T[h(k)] instead of T[k].

This immediately creates a problem: two different keys k_1 and k_2 can hash to the same slot $h(k_1) = h(k_2)$. This is

called a collision. Since $|U| >> m$, by the pigeonhole

principle, collisions are inevitable if we store more than m elements but they can happen even with far fewer elements due to the birthday paradox.

The two problems a hash table must solve:

i. Design hash functions that distribute keys uniformly across slots, minimizing collision probability.
ii. Handle collisions when they occur.

What Makes a Good Hash Function

A good hash function satisfies simple uniform hashing each key is equally likely to hash to any of the m slots, independently of where other keys hash. This is an ideal we approximate in practice.

In reality we cannot guarantee simple uniform hashing because we do not know the input distribution in advance. But we can design hash functions that behave well for most inputs.

The key requirements:

The function must be fast to compute O(1).

It must distribute keys as uniformly as possible across the m slots. It must be deterministic (same key always maps to same slot).

<u>The Division Method</u>

$h(k) = k \bmod m$

Simple and fast. The value of k modulo m is always in [0, m-1].

Choice of m matters critically. Bad choices of m cause clustering:

If m is a power of 2, h(k) = k mod 2p is just the last p bits of k. If keys share common low-order bit patterns, many keys collide. Avoid powers of 2.

If m = 10r, h(k) depends only on the last r decimal digits of k. Again, low-order digits may not be uniformly distributed.

Good choice: m is a prime not too close to a power of 2. For example, if you need a table of approximately 1000 slots, use m = 997.

Weakness: For certain key distributions, even a prime m can produce poor distribution.

The division method is sensitive to the relationship between the key distribution and m.

The Multiplication Method:

$h(k) = \lfloor m \cdot (k \cdot A \bmod 1) \rfloor$ where A is a constant, $0 < A < 1$.

Why this works better: The fractional part of k·A involves all bits of k, not just the low-order bits. The result is more uniformly distributed.

Choice of A: Knuth recommends $A = (\sqrt{5} - 1)/2 \approx 0.6180339887$ the reciprocal of the golden ratio. This choice has good empirical properties.

Advantage over division method: m does not need to be prime. A power of 2 is fine in fact, when m = 2p the computation is particularly fast using bit operations.

Practical implementation: If w is the word size (e.g., 32 bits), choose A =s/2w for some integer s. Then k·A mod 1 is computed by multiplying k by s (integer multiplication), and taking the upper p bits of the lower w bits of the result.

Collision Resolution by Chaining

The simplest collision resolution strategy: each slot of the hash table holds a linked list of all elements that hash to that slot.

Insert(T, x):

prepend x to list T[h(x.key)] ←O(1)

Search(T, k):

search list T[h(k)] for key k ←O(length of list)

Delete(T, x):

delete x from list T[h(x.key)] ← O(1) with doubly linked list

Insert is O(1) always prepend to the front.

Delete is O(1) with doubly linked lists (given a pointer to the element).

Search takes time proportional to the length of the list at T[h(k)].

Analysis Under Simple Uniform Hashing

Load factor: $\alpha = n/m$ the average number of elements per slot. n is the number of elements stored, m is the number of slots.

Under simple uniform hashing, the expected length of each list is α.

Unsuccessful search: We must scan the entire list at T[h(k)].

Expected length is α. Expected time: $\Theta(1 + \alpha)$.

The $\Theta(1)$ accounts for computing h(k). The α accounts for scanning the list.

Successful search: We find the element partway through the list. By a symmetry argument (the searched-for element is equally likely to be any element in the list), expected

comparisons is $1 + \alpha/2$ (1 for the found element plus $\alpha/2$ for elements inserted after it on average, since we prepend).

Expected time: $\Theta(1 + \alpha)$.

The critical insight: If $n = O(m)$ (the number of elements is proportional to the table size) then $\alpha = n/m = O(1)$, and all operations take O(1) expected time.

This is the hash table's fundamental promise: O(1) expected time for search, insert, and delete when the load factor is O(1). The key qualifier is expected averaged over the hash function's behavior on the input. Worst case (all elements hash to one slot) is O(n).

The Birthday Paradox and Why Collisions

Are Inevitable:

If n elements are inserted into a table of m slots with a uniform hash function, the probability that at least one collision occurs is approximately:

$$1 - e^{-n^2/2m}$$

For m = 365 (days in a year) and n = 23 people, this probability exceeds 0.5.

That is the birthday paradox: with only 23 people in a room, there is a better than 50% chance two share a birthday.

For hash tables: if m = 106 and n = 1000, the collision probability is approximately 1– e-500 ≈ 1. With n = √(2m) ≈ 1414 elements, collision probability exceeds 0.5.

Collisions happen much sooner than intuition suggests. Any honest hash table design must handle them avoiding collisions is not a realistic goal.

Dynamic Resizing

As elements are inserted, α = n/m grows. When α exceeds some threshold (typically 0.7-0.75 for open addressing, up to 1 or beyond for chaining), performance degrades. The

solution is dynamic resizing rebuild the hash table with a larger m.

The process: Allocate a new table with m' ≈ 2m slots. Recompute h'(k) for every existing key (the new hash function uses m', so all old positions are invalid).

Insert every element into the new table.

Amortized cost: Rebuilding costs Θ(n) proportional to the number of elements. But it happens infrequently only when n reaches m, then when n reaches 2m, and so on. The

total rebuild cost for n insertions is n + n/2 + n/4 + ... = 2n = O(n). Spread across n insertions, the amortized cost per insertion is O(1).

The constant factor 2 for table size growth ensures geometric spacing between rebuilds and O(1) amortized insert.

Similarly, when deletions cause n to fall below m/4, shrink the table to m/2. This prevents wasting space while maintaining O(1) amortized cost.

Open Addressing

In open addressing, all elements are stored directly in the hash table no linked lists, no pointers.

When a collision occurs, we probe alternative slots according to a probe sequence until we find an empty slot.

The table can store at most m elements the load factor $\alpha = n/m$ must be strictly less than 1.

The probe sequence: Instead of a single hash value h(k), we define a probe sequence h(k, 0), h(k, 1), h(k, 2), ..., h(k, m-1) a permutation of all m slots.

We try them in order until we find an empty slot (for insert) or the key (for search).

Open Addressing

Insert(T, k):

for i = 0 to m-1:

j = h(k, i)

if T[j] == NIL or T[j] == DELETED:

T[j] = k

return j

error "hash table overflow"

Search(T, k):

for i = 0 to m-1:

j = h(k, i)

if T[j] == NIL: return NIL ← key not present

if T[j] == k: return j

return NIL

The DELETED marker: When deleting, we cannot simply set T[j] = NIL that would break search chains. A search for a key that was inserted after the deleted slot would stop prematurely at the NIL. Instead, mark deleted slots with a special DELETED sentinel. Search skips DELETED slots. Insert can reuse them. This complicates analysis slightly.

Linear Probing

h(k, i) = (h'(k) + i) mod m

Start at h'(k), probe consecutively: h'(k), h'(k)+1, h'(k)+2, ...

Simple and cache-friendly consecutive probes access adjacent memory locations.

The clustering problem: Linear probing suffers from primary clustering. Long runs of occupied slots accumulate any key that hashes into a run extends it. The longer the run, the more likely new insertions land in it, making it longer still. A positive feedback loop.

Expected probe length with linear probing for unsuccessful search: approximately $(1 + 1/(1-\alpha)^2) / 2$ grows as $1/(1-\alpha)^2$ as α approaches 1.

For $\alpha = 0.9$, this is about 50 probes.

For $\alpha = 0.99$, about 5000 probes.

Linear probing degrades badly at high load factors.

Quadratic Probing

$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$

For appropriately chosen constants c_1, c_2 and prime m, this generates all m slots.

Avoids **primary clustering** the probe sequence jumps around rather than scanning linearly.

But suffers from **secondary clustering:**

keys with the same h'(k) value follow identical probe sequences.

Two keys with the same initial hash always collide at the same sequence of slots.

Double Hashing

$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$

Two independent hash functions. The step size $h_2(k)$ is different for each key, so secondary clustering is eliminated.

For all m slots to be probed, $h_2(k)$ must be coprime to m.

A commonchoice: m is prime, $h_2(k) = 1 + (k \bmod (m-1))$ always in [1, m-1], always coprime to prime m.

Double hashing gives the best distribution among the three linear methods and is the closest to truly independent probing.

Analysis of Open Addressing

Under uniform hashing**:**

Expected probes for unsuccessful search: $\leq 1/(1-\alpha)$

Expected probes for successful search: $\leq (1/\alpha) \cdot \ln(1/(1-\alpha))$

For $\alpha = 0.5$: unsuccessful ≤ 2 probes, successful ≤ 1.4 probes. Excellent.

For $\alpha = 0.9$: unsuccessful ≤ 10 probes, successful ≤ 2.6 probes. Still manageable.

For $\alpha = 0.99$: unsuccessful ≤ 100 probes, successful ≤ 4.7 probes. Painful.

The lesson: keep α below 0.7-0.8 in practice.

Resize before the table gets too full.

Universal Hashing

Everything so far assumes either simple uniform hashing or uniform hashing — idealizations that we cannot actually guarantee. A determined adversary who knows your hash function can craft inputs where every key hashes to the same slot, causing O(n) operations. This is not hypothetical — hash-flooding attacks on web servers exploited exactly this vulnerability.

Universal hashing is the solution. Instead of a fixed hash function, choose a hash function randomly from a carefully designed family. No adversary who does not know which

function was chosen can craft a bad input.

Definition: A collection H of hash functions from U to {0, ..., m-1} is universal if for any two distinct keys $k_1, k_2 \in U$:

$Pr[h(k_1) = h(k_2)] \leq 1/m$

where the probability is over the random choice of h from H.

The collision probability equals the collision probability for truly random functions — 1/m.

No pair of keys collides more often than this, regardless of what those keys are.

A Universal Family

Let m be prime. Decompose each key k as a sequence of r+1 digits in base m:

$k = \langle k_0, k_1, ..., k_r \rangle$ where each $k_i \in \{0, ..., m-1\}$.

Choose a random vector $a = \langle a_0, a_1, ..., a_r \rangle$ where each a_i is chosen uniformly at random from {0, ..., m-1}.

Define:

$h\_a(k) = (\sum a_i \cdot k_i) \bmod m$

The family $H = \{h\_a : a \in \{0,...,m-1\}^{(r+1)}\}$ is universal.

Proof sketch: For any two distinct keys k and k', they differ in at least one position, say position j. For a randomly

chosen a, the event $h\_a(k) = h\_a(k')$ reduces (by algebraic manipulation mod prime m) to a_j being a specific value

given the other a_i. Since a_j is uniform in {0,...,m-1}, the probability is exactly 1/m.

Performance Guarantee

Theorem: Using a universal hash function family with chaining, the expected number of collisions for any key k is at most $\alpha = n/m$ (the load factor), regardless of the input.

Proof: For a fixed key k, let $X_{kj} = 1$ if key j collides with k, 0 otherwise. $E[X_{kj}] = Pr[h(k) = h(j)] \leq 1/m$ by universality. Total expected collisions with k: $\sum E[X_{kj}] \leq n/m = \alpha$.

This is the critical result. With universal hashing, we no longer need any assumption about the input distribution. The expected O(1) performance guarantee holds for any input — because the randomness is in the hash function choice, not the input.

This is why modern language runtime systems (Python dictionaries, Java HashMaps) use randomized hash function seeds. When you restart a Python process, the hash function changes. An adversary who crafted a bad input for one run cannot predict what the bad input is for the next run.

Perfect Hashing

The setting: All n keys are known in advance and fixed. No insertions or deletions after construction.

The goal: O(1) worst-case search — not expected, not amortized, but guaranteed worst case.

The idea: Two-level hashing with universal hash functions.

Level 1: Hash all n keys to m = n slots using a universal hash function h. Some slots will have collisions — let n_i be the number of keys hashing to slot i.

Level 2: For each slot i with $n_i > 1$, build a secondary hash table of size $m_i = n_i^2$ using another universal hash function h_i, chosen such that no two keys in slot i collide in the

secondary table.

Whyn_i^2? With a secondary table of size n_i^2, a random universal hash function has expected number of collisions among the n_i keys of at most $C(n_i, 2)/n_i^2 = (n_i(n_i-1)/2)/n_i^2 < 1/2$. So the probability of any collision is less than 1/2. If we get a collision, pick another hash function —expected 2 tries before finding a collision-free one

Search: Compute h(k) → slot i. Compute $h_i(k)$ → position in secondary table. One comparison. O(1) worst case.

Space analysis: Expected total secondary table space = $E[\sum n_i^2]$.

Using the collision analysis: $E[\sum n_i^2] = E[\sum n_i(n_i-1)] + n \leq n + n = 2n = O(n)$.

The key step uses the fact that $\sum n_i(n_i-1)$ is twice the expected number of collisions at level 1, which with a universal family on a table of size n is at most $C(n,2)/n = (n-1)/2 < n/2$. So total expected space is O(n).

Construction time: O(n) expected — linear passes to build both levels, expected constant number of hash function retries at level 2.

Perfect hashing achieves: O(1) worst-case search, O(n) space, O(n) expected construction time. The price is that it only works for static key sets.

Hash Tables vs. Red-Black Trees

This comparison comes up constantly in practice and is worth addressing directly.

Use a hash table when: You need O(1) average search/insert/delete and do not care about order. The key set is not adversarially chosen (or you use universal hashing).

You do not need successor, predecessor, rank, or range queries. Python dicts, JavaScript objects, database hash indexes are all hash tables.

Use a Red-Black Tree when: You need O(log n) worst-case guaranteed (not just expected). You need order-based operations — minimum, maximum, successor, predecessor, range queries, sorted iteration. You cannot afford the occasional O(n) worst case even with negligible probability. C++ std::map, Java TreeMap, Linux scheduler are all Red-Black Trees.

The fundamental tradeoff: Hash tables trade order for speed. Red-Black Trees sacrifice O(1) average performance for O(log n) worst-case guarantee and full order support.

Scheme	Search (expected)	Search (worst)	Space	Notes
Chaining (SUHA)	O(1+α)	O(n)	O(n+m)	Simple, handles α > 1
Open addressing (linear)	O(1/(1-α))	O(n)	O(m)	Clustering problem
Open addressing (double hash)	O(1/(1-α))	O(n)	O(m)	Best open addressing
Universal + chaining	O(1+α) expected	O(n)	O(n+m)	No input assumptions
Perfect hashing	O(1)	O(1)	O(n)	Static keys only

Synthesis

The progression is clear. SUHA gives expected O(1) under an assumption. Universal hashing removes the assumption. Perfect hashing achieves O(1) worst case at the cost of static keys. Each step gives a stronger guarantee at some additional cost in complexity or flexibility.

(10)Quicksort

The Algorithm

Quicksort(A, lo, hi):

if lo < hi:

p =Partition(A, lo, hi)

Quicksort(A, lo, p-1)

Quicksort(A, p+1, hi)

Quicksort is divide-and-conquer. The divide step is Partition it rearranges the array so that:

One element (the pivot) is in its final sorted position p

All elements left of p are ≤ pivot

All elements right of p are ≥ pivot

The **conquer step** sorts the two subarrays recursively.

The **combine step** is nothing because Partition places the pivot exactly where it belongs, and the subarrays are sorted independently, the whole array is sorted when the recursion completes.

This is the opposite of **Merge Sort: Merge Sort** does trivial divide work and expensive combine work.

Quicksort does expensive divide work (Partition) and trivial combine work (nothing).

The Partition Algorithm

The Lomuto partition scheme:

Partition(A, lo, hi):

pivot = A[hi] ←choose last element as pivot

i = lo- 1 ←i tracks the boundary of elements ≤ pivot

for j = lo to hi- 1:

if A[j] ≤ pivot:

i = i + 1

swap A[i] and A[j]

swap A[i+1] and A[hi] ←place pivot in its correct position

return i + 1 ←return pivot's final index

Loop invariant:

At the start of each iteration, A[lo..i] contains elements ≤ pivot, A[i+1..j 1] contains elements > pivot, and A[j..hi-1] is unexamined.

When j reaches hi (the pivot), A[lo..i] ≤ pivot and A[i+1..hi-1] > pivot.

Swapping A[i+1] with A[hi] places the pivot at position i+1 its correct final position.

Time complexity of Partition: Scans every element exactly once.

Θ(n) for an array of size n.

Space: In-place. Only a constant number of variables.

Worst Case $O(n^2)$

The worst case occurs when Partition consistently produces maximally unbalanced

splits one subarray of size n-1 and one of size 0.

This happens when the pivot is always the smallest or always the largest element.

With the Lomuto scheme using the last element as pivot, this occurs when the input is already sorted (ascending or descending).

The **recurrence** for worst-case Quicksort:

$T(n) = T(n-1) + T(0) + \Theta(n) = T(n-1) + \Theta(n)$

This is the subtractive recurrence. Solving by telescoping:

$T(n) = T(n-1) + cn = T(n-2) + c(n-1) + cn = ... = c(1 + 2 + ... + n) = \Theta(n^2)$

This is catastrophic. Sorting an already-sorted array with naive Quicksort takes quadratic time. Since sorted or nearly-sorted input is extremely common in practice, this is a real problem, not just a theoretical curiosity.

Best Case O(n log n)

The best case occurs when Partition always splits the array exactly in half pivot lands at the midpoint every time.

$T(n) = 2T(n/2) + \Theta(n)$

By the Master Theorem Case 2: $\Theta(n \log n)$.

Average Case O(n log n)

We assume all n! permutations of the input are equally likely.

Quicksort runs in $\Theta(n \log n)$ time on average

While a perfect 50:50 split is ideal, it rarely happens.

Even remarkably unbalanced splits (like a constant 9:1 or 99:1 ratio) still produce an optimal $\Theta(n \log n)$ runtime.

In the real world, random inputs naturally balance out over the course of the recursion tree, preventing the algorithm from collapsing into its quadratic worst case.

The Perfect Split (1:1 Ratio)

Recurrence: $T(n) = 2T(n/2) + \Theta(n)$

Tree Depth: $\log_2 n$

Total Cost: $\Theta(n \log n)$

The Unbalanced Split (9:1 Ratio)

Recurrence: $T(n) = T(n/10) + T(9n/10) + \Theta(n)$

Tree Depth:

$\log_{10/9} n \approx 2.26 \log_2 n$ (The tree is slightly deeper, but still logarithmic!)

Total Cost: $\Theta(n \log n)$

Randomized Quicksort

The weakness of deterministic Quicksort is that adversarial inputs (sorted arrays, reverse-sorted

arrays) trigger worst-case behavior. The fix is simple and powerful: randomly choose the pivot.

RandomizedPartition(A, lo, hi):

i = Random(lo, hi) ←uniformly random index

swap A[i] and A[hi] ←putrandomelement in pivot position

return Partition(A, lo, hi)

RandomizedQuicksort(A, lo, hi):

if lo < hi:

p = RandomizedPartition(A, lo, hi)

RandomizedQuicksort(A, lo, p-1)

RandomizedQuicksort(A, p+1, hi)

Why this works:

By choosing the pivot randomly, no specific input can consistently trigger worst-case behavior.

The expected running time of Randomized Quicksort is $\Theta(n \log n)$ for any input.

The worst case is still technically $O(n^2)$ there is a small probability that random choices consistently pick bad pivots.

But this probability is astronomically small. For n = 106, the probability that Randomized Quicksort takes more than 10× its expected time is less than 1/n essentially impossible in practice.

Practical Improvements

Production implementations of Quicksort use several refinements:

Median-of-three pivot: Instead of a single random pivot, choose three random elements and use their median. This improves the expected split quality and reduces the probability of bad cases.

Three-way partition (Dutch National Flag): Handles duplicate elements efficiently. Standard partition degrades on arrays with many duplicates. Three-way partition divides the array into three regions: < pivot, = pivot, > pivot. Elements equal to the pivot are already in their final positions and excluded from further recursion. This makes Quicksort O(n) on arrays with all identical elements.

Hybrid with Insertion Sort: For small subarrays (n ≤ about 10-20), Insertion Sort is faster than Quicksort due to lower overhead. Production Quicksort implementations switch to Insertion Sort below a threshold.

Introsort: The algorithm used by C++ std::sort. Starts with Quicksort, monitors recursion depth, and switches to Heapsort if depth exceeds 2 log n. This guarantees O(n log n) worst case while preserving Quicksort's practical speed for typical inputs.

(11)Binary Search Trees:

Structure and the BST Property

A Binary Search Tree is a binary tree where each node contains a key and satisfies:

BST Property: For any node x, all keys in x's left subtree are ≤ key[x],

and all keys in x's right subtree are ≥ key[x].

Each node typically stores: key, left child pointer, right child pointer, parent pointer, and any satellite data.

The BST property means the structure encodes order. Keys to the left are smaller, keys to the right are larger.

This is what enables efficient search at each node, a comparison tells you which subtree the target must be in.

In-order traversal gives sorted output:

InorderTraversal(x):

if x ≠ NIL:

InorderTraversal(x.left)

print x.key

InorderTraversal(x.right)

This visits every node exactly once:

Θ(n). The output is the keys in sorted order.

Search

Search(x, k):

if x == NIL or k == x.key:

return x

if k < x.key:

return Search(x.left, k)

else:

return Search(x.right, k)

At each node, one comparison determines which half of the remaining tree to explore. The number of comparisons is bounded by the height h of the tree.

Time: O(h) where h is the tree height.

Minimum (leftmost node)

Successor (next larger key)

Maximum(rightmost node)

Predecessor (next smaller key).

All O(h).

Insertion

Insert(T, z):

y = NIL, x = T.root

while x ≠ NIL: ←find insertion point

y =x

if z.key < x.key: x = x.left

else: x = x.right

z.parent = y

if y == NIL: T.root = z ←tree was empty

elif z.key < y.key: y.left = z

else: y.right = z

Walk down the tree following BST property until you fall off an edge. Insert the new node there. Time: O(h).

Deletion

Deletion is the most complex BST operation. Three cases:

Case 1 Node z has no children: Simply remove z. Set parent's pointer to NIL.

Case 2 Node z has one child: Replace z with its child. Set parent's pointer to z's child.

Case 3 Node z has two children: Find z's successor the node with the smallest key greater than z (leftmost node of z's right subtree). Replace z's key with the successor's key, then delete the successor (which has at most one child (its right child)so this is Case 1 or Case 2).

Time: O(h).

The Height Problem

All BST operations run in O(h). What is h?

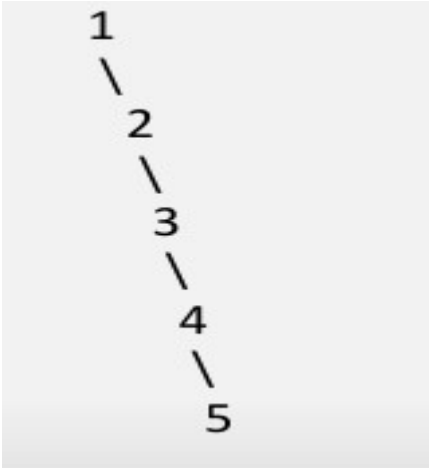
Best case perfectly balanced tree: h = Θ(log n). All operations O(log n).

This is ideal.

Worst case completely unbalanced (degenerate) tree: h = Θ(n). All operations O(n). This is no better than a linked list.

When does the worst case happen? When elements are inserted in sorted order. Each new element is larger than all existing ones, goes to the right, and the tree becomes a right-going chain.

Insert 1, 2, 3, 4, 5 in order → **tree looks like:**



The Height Problem

Height 4 for 5 elements. In general, h = n-1.

Average case height: If keys are inserted in a uniformly random order, the expected height is Θ(log n). The expected cost of operations is O(log n). This is analogous to the average-case analysis of Quicksort random permutations lead to balanced-ish structure.

But just like **Quicksort** can degrade on sorted input, BSTs degrade on sorted insertion order. We cannot rely on randomness of input.

Treesort and Its Connection to Quicksort

Treesort: Insert all n elements into a BST, then extract them with in

order traversal.

Insert n elements: O(n·h). In-order traversal: Θ(n). Total: O(n·h).

If the tree is balanced: O(n log n). If the tree is degenerate: $O(n^2)$.

This is not a coincidence. The comparisons made by Treesort on a random permutation are exactly the comparisons made by Quicksort with the same first element as pivot.

The BST structure and the Quicksort recursion tree are isomorphic. The expected height of a randomly built BST is exactly the expected depth of recursion in Quicksort, both Θ(log n), for the same reason.

(12)Red-Black Trees

Red-Black tree

A red-black tree is a binary search tree with one extra bit of storage per node: its color, which can be either RED or BLACK.

By constraining the node colors on any simple path from the root to a leaf, red-black trees ensure that no such path is more than twice as long as any other, so that the tree is approximately balanced.

Red-Black Properties

A red-black tree is a binary tree that satisfies the following red-black properties:

Property 1: Every node is either red or black.

Property 2: The root is black.

Property 3: Every leaf (NIL) is black.

Property 4: If a node is red, then both its children are black. (No two consecutive red nodes on any path.)

Property 5: For each node, all simple paths from that node to any descendant leaf contain the same number of black nodes.

The number of black nodes on any path from a node to a descendant leaf is called the node's black-height, denoted bh(x).

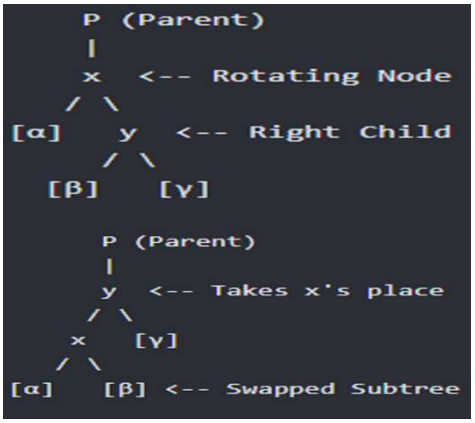
Property 5 says black-height is well-defined every path has the same number of black nodes. This uniformity is what prevents extreme imbalance.

NIL nodes (the sentinels) are treated as black leaves with no key.

Rotations

Insertions and deletions can violate the Red-Black properties.

To restore them, we use rotations local restructuring operations that change the tree's shape while preserving the BST property.



Left Rotation on node x:

x's right child y takes x's position.

x becomes y's left child.

y's former left child becomes x's right child.

Rotations

LeftRotate(T, x):

y = x.right

x.right = y.left

if y.left ≠ T.nil: y.left.parent = x

y.parent = x.parent

if x.parent == T.nil: T.root = y

elif x == x.parent.left: x.parent.left = y

else: x.parent.right = y

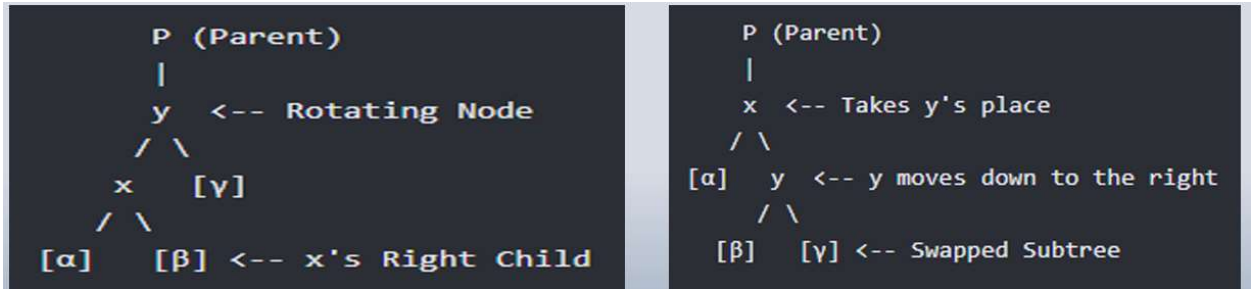
y.left = x

x.parent = y

Right Rotation on y: The mirror image.

y's left child x takes y's position.

Rotations run in O(1) they update a constant number of pointers. They preserve the BST property because they maintain the left-less-than, right-greater-than relationship at every node affected.



Red-Black Insertion

Insert as in a regular BST, color the new node red, then fix any violations.

RBInsert(T, z):

[BST insert z, set z.color = RED]

RBInsertFixup(T, z)

Why red? Inserting a black node would immediately violate Property 5 on the path containing it (adding one black node to those paths).

Inserting red might violate Property 4 (two consecutive reds) but at least does not violate Property 5.

RBInsertFixup fixes Property 4 violations. The violation exists only at z and z's parent (both red). There are three cases based on the color of z's uncle (z's

parent's sibling):

Case 1 **Uncle is red:** Recolor: parent and uncle become black, grandparent becomes red. Move the violation up to the grandparent and continue.

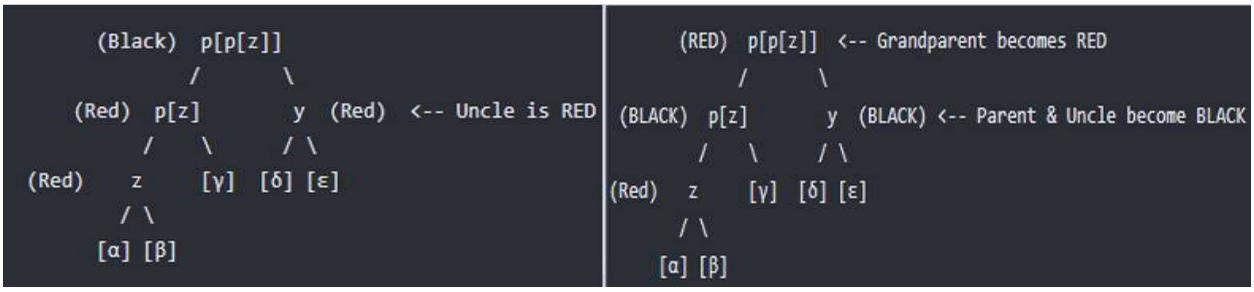
G(B) G(R) ←might now violate with G's parent

/ \ / \

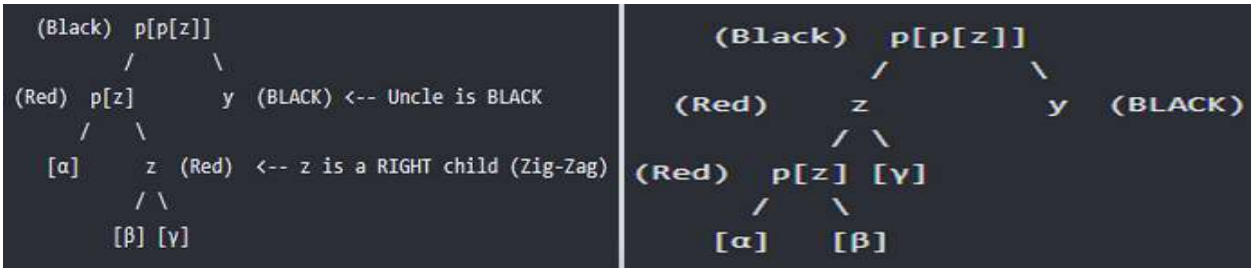
P(R) U(R) → P(B) U(B)

| |

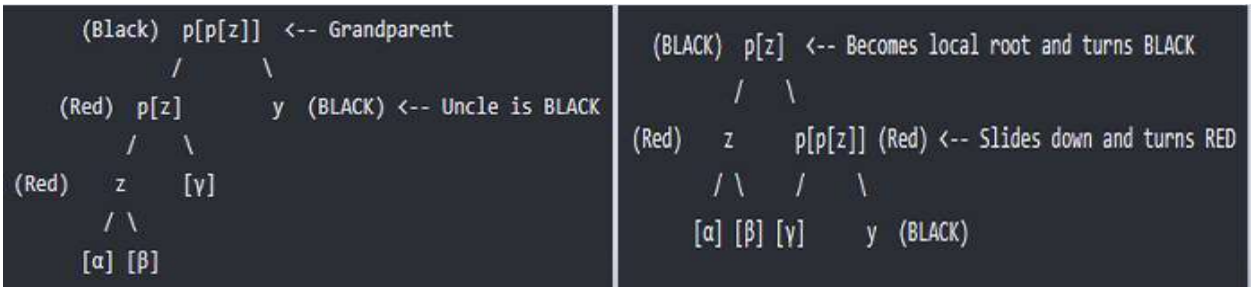
z(R) z(R) ←violation resolved locally



Case 2 (Zig-Zag) Uncle is black, z is a right child: Left rotate on parent to convert to Case 3.



Case 3 (Straight Line) Uncle is black, z is a left child: Right rotate on grandparent, recolor. The violation is eliminated.



Red-Black Insertion

Number of rotations:

At most 2 rotations total per insertion.

Case 1 does not rotate it recolors and moves up.

Cases 2 and 3 each do 1 rotation and terminate.

So insertion does at most 2 rotations.

Time: O(log n)

O(log n) for BST insert, O(log n) for fixup (Case 1 can propagate up

O(log n) levels but does no rotations,

Cases 2 and 3 terminate immediately).

Red-Black Deletion

Deletion is substantially more complex than insertion.

After deleting a node, we may have violated the Red-Black properties.

The fixup procedure RBDeleteFixup handles four cases, depending on the color of the sibling of the node that was removed and its children's colors.

Key facts:

Deletion may require up to 3 rotations.

Time: O(log n).

All five properties are restored after fixup.

Performance Summary

All operations guaranteed O(log n) for any sequence of operations on any input. This is the payoff for the complexity of maintaining the Red-Black invariants.

Red-Black Trees are used in: Linux kernel process scheduler (completely fair scheduler), C++ std::map and std::set, Java TreeMap and TreeSet, many database index structures.

Operation	Time
Search	O(log n)
Minimum / Maximum	O(log n)
Successor / Predecessor	O(log n)
Insert	O(log n)
Delete	O(log n)

(13)Greedy Approach

Optimization Problem

An optimization problem is any scenario where you must choose the absolute best solution out of a large pool of possible combinations under strict rules or limits.

To solve it, you must navigate a set of constraints while trying to achieve a perfect final score. This score is evaluated by an objective function, which serves as your measuring stick for success.

The goal of optimization always falls into one of two objectives:

Maximization: You want to push your score to the highest possible limit, such as maximizing profit margins.

Minimization: You want to drive your costs or waste down to the lowest possible floor, such as finding a route that minimizes total travel time.

Greedy Algorithm

A greedy algorithm builds a solution piece by piece, and at each step it makes the choice that looks best at that moment without looking ahead, without reconsidering past choices, and without worrying about the global consequences of the local decision.

Brute Force tries every possible solution and picks the best. Correct, but often exponentially slow.

Divide and Conquer breaks the problem into independent subproblems, solves each recursively, then combines. The subproblems are genuinely separate (solving one doesn't affect another).

Greedy makes one choice at a time, each choice narrowing down the remaining problem. Once a choice is made, it is never reconsidered. There is no backtracking.

Dynamic Programming also makes choices, but it considers all possible choices at each step and remembers the results. It's greedy's more careful sibling (slower but applicable to a wider class of problems).

The two properties that a problem must have for a greedy algorithm to be correct.

If both hold, **greedy** works. If either is absent, greedy may fail.

Property 1: Greedy Choice Property

A globally optimal solution can be arrived at by making locally optimal (greedy) choices.

This means: there exists an optimal solution that includes the greedy choice. You don't have to consider any alternative. You can safely commit to the locally best

option and know that some optimal solution is consistent with that commitment.

This is the property that must be proven for every greedy algorithm.

The proof structure is almost always the same: assume an optimal solution exists that does not include the greedy choice, then show you can modify it to include the greedy choice without making it worse. This exchange argument is the standard proof technique for greedy correctness.

Property 2: Optimal Substructure

An optimal solution to the problem contains within it optimal solutions to subproblems.

After making the greedy choice and reducing the problem, the remaining subproblem must also be solvable optimally by the same greedy strategy. The greedy choice doesn't just work once (it works repeatedly, all the way down).

Both DP and greedy exploit this property. The difference is that greedy additionally has the greedy choice property, which lets it commit to one choice without exploring alternatives. DP lacks this luxury and must consider all choices.

<u>Activity Selection Problem</u>:

Setting: You have a single resource (a lecture hall, a

processor, a machine). You have n activities, each wanting to use this resource during a specific time interval. Activity i has a start time s_i and a finish time f_i, where $s_i < f_i$.

Two activities i and j are compatible if their intervals don't overlap: either $f_i \leq s_j$ or $f_j \leq s_i$ (one finishes before the other starts).

Goal: Select the maximum number of mutually compatible activities.

Note carefully: this is a maximization problem. We want as many activities as possible, not the activities with the longest duration or highest value. Just the most activities.

A Concrete Example

Suppose we have 11 activities (indexed 1 to 11), already sorted by finish time:

Activity	1	2	3	4	5	6	7	8	9	10	11
Start s_i	1	3	0	5	3	5	6	8	8	2	12
Finish f_i	4	5	6	7	9	9	10	11	12	14	16

We want the largest set of non-overlapping activities.

One optimal solution is: {1, 4, 8, 11} activities finishing at times 4, 7, 11, 16.

Another optimal solution is: {1, 4, 9, 11}.

Both have 4 activities. No set of 5 mutually compatible activities exists for this input.

Thinking About Greedy Choices

Before committing to an algorithm, ask: what greedy strategy should we try?

Several candidates present themselves:

Idea 1: Always pick the activity that starts earliest.

Intuition: start early, leave maximum room for others.

Counterexample: an activity starting at time 0 but finishing at time 100 blocks everything else.

Idea 2: Always pick the shortest activity.

Intuition: short activities consume less time, leave more room.

Counterexample: a short activity placed in the middle can block two compatible activities on either side.

Idea 3: Always pick the activity with the fewest conflicts.

Intuition: choose activities that interfere least.

Counterexample: constructing one is more involved, but this greedy fails on certain inputs.

Idea 4: Always pick the activity that finishes earliest (among compatible ones).

Intuition: finishing early leaves the maximum remaining time for future activities. This one works.

The Greedy Algorithm

GREEDY-ACTIVITY-SELECTOR(s, f)

// Assumes activities sorted by finish time: f[1] ≤ f[2] ≤ ... ≤ f[n]

1 n = s.length

2 A = {1} // Always select the first activity

3 k = 1 // k tracks the last activity added

4 for m = 2 to n

5 if s[m] ≥ f[k] //Activity m starts after last selected finishes

6 A = A ∪ {m}

7 k = m

8 return A

Tracing through our example:

Start: A = {1}, k = 1, f[1] = 4

m=2: s[2]=3 < f[1]=4 → skip

m=3: s[3]=0 < 4 → skip

m=4: s[4]=5 ≥ 4 → add. A={1,4}, k=4, f[4]=7

m=5: s[5]=3 < 7 → skip

m=6: s[6]=5 < 7 → skip

m=7: s[7]=6 < 7 → skip

m=8: s[8]=8 ≥ 7 → add. A={1,4,8}, k=8, f[8]=11

m=9: s[9]=8 < 11 → skip

m=10: s[10]=2 < 11 → skip

m=11: s[11]=12 ≥ 11 → add. A={1,4,8,11}, k=11

Result: {1, 4, 8, 11} 4 activities. Optimal.

Running Time

If activities are pre-sorted by finish time: $\Theta(n)$ (a single pass through the array).

If we must sort first: $\Theta(n \log n)$ (dominated by sorting).

Proving Correctness: The Exchange Argument

We must prove the greedy choice (always selecting the activity with the earliest finish time) is safe.

Activity Selection Problem

Theorem: Consider any non-empty subproblem S_k (activities compatible with all previously selected activities). Let activity m be the activity in S_k with the earliest finish time. Then m is included in some maximum-size subset of mutually compatible activities from S_k.

Proof:

Let A* be a maximum-size subset of mutually compatible activities from S_k.

Let activity j be the activity in A* with the earliest finish time (the first activity in A*).

Case 1: j = m. Then m is already in A*. Done.

Case 2: $j \neq m$. Since m has the earliest finish time in S_k and j is also in S_k:

$fm \leq fj$

Construct A' = (A* \ {j}) ∪ {m} replace j with m in the optimal solution.

Is A' still valid (mutually compatible)?

m is compatible with all activities in A* that come before j: there are none, since j is the first activity in A*.

m is compatible with all activities in A* that come after j: since $f_m \leq f_j \leq s_l$ for any activity l after j, m finishes no later than j did, so m doesn't overlap any later activity.

|A'| = |A*| since we replaced one activity with another.

So A' is also a maximum-size subset that includes m.

This exchange argument shows: we never lose anything by choosing the earliest-finishing activity. Any optimal solution that doesn't include it can be transformed into one that does, without reducing the count. The greedy choice is always safe.

Optimal Substructure

After choosing activity m (earliest finish), the remaining problem is:

find the maximum set of compatible activities from those that start at or after f_m. This is the same problem structure on a smaller input. The greedy strategy applies again identically. By induction, the algorithm is correct all the way through.

Huffman Coding

Huffman coding is one of the most elegant and practically important greedy algorithms ever devised. It is the foundation of file compression formats including

ZIP, JPEG, MP3, and PNG.

The setting: You have a file consisting of characters drawn from an alphabet C. Each character $c \in C$ appears with frequency f(c). You want to represent the file in

binary, assign each character a binary string (a code word) such that the total number of bits used to encode the file is minimized.

Two types of codes:

Fixed-length encoding: Every character gets the same length codeword.

For an alphabet of size |C|, you need $\lceil \log_2 |C| \rceil$ bits per character. Simple, but wasteful (frequent characters get the same space as rare ones).

Variable-length encoding: Frequent characters get short codewords; rare characters get long ones. This is the idea Huffman exploits.

The Prefix-Free Requirement:

For variable-length codes to be unambiguously decodable, we need the

prefix-free property: no codeword is a prefix of any other codeword.

For example: if 'a' = 0 and 'b' = 01, then seeing a 0 is ambiguous is it 'a', or the start of 'b'? Prefix-free codes eliminate this ambiguity. With a prefix-free code, you read bits left-to-right and as soon as you hit a complete codeword, you decode it (no look ahead needed).

Prefix-free codes correspond exactly to leaves of a binary tree:

Each character sits at a leaf, and its codeword is the path from root to that leaf (0 = go left, 1 = go right).

Since no leaf is an ancestor of another leaf, the prefix-free property is automatically satisfied.

The Cost of an Encoding Tree

Given a binary tree T representing a prefix-free code, the total number of bits to encode the file is:

$$B(T) = \sum_{c \in C} f(c) \cdot d_T(c)$$

where dT(c) is the depth of character c in the tree (= length of its codeword).

We want to find the tree T that minimizes B(T).

Huffman Coding

Example

Suppose we have 6 characters with frequencies:

Character	a	b	c	d	e	f
Frequency	45	13	12	16	9	5

(Frequencies are in thousands, or think of them as percentages summing to 100.)

Fixed-length encoding: 6 characters need 3 bits each.

Total bits = 3 × 100 = 300 thousand bits.

Optimal variable-length (Huffman) encoding:

Achieves 224 thousand bits, a 25% reduction.

Huffman's Algorithm

The key insight Huffman had : the two least frequent characters should be deepest in the tree (longest codewords), and they should be siblings (sharing a parent node).

HUFFMAN(C)

1 n=|C|

2 Q=C //Build a min-priority queue by frequency

3 for i=1ton-1

4 allocate new node z

5 z. left=x=EXTRACT-MIN(Q)

6 z. right=y=EXTRACT-MIN(Q)

7 z. freq =x. freq+y.freq

8 INSERT(Q,z)

9 returnEXTRACT-MIN(Q) //Root of the Huffman tree

The algorithm repeatedly:

Extracts the two nodes with the lowest frequencies.

Creates a new internal node with these two as children.

Assigns the new node a frequency equal to the sum of its children's frequencies

Inserts the new node back into the queue

This continues until only one node remains (the root of the Huffman tree).

Huffman Coding

Tracing Through the Example

Initial queue (sorted by frequency): f:5, e:9, c:12, b:13, d:16, a:45

Iteration 1:

Extract f(5) and e(9). Create z_1 with freq 14. Queue: c:12, b:13, z_1:14, d:16, a:45

Iteration 2:

Extract c(12) and b(13). Create z_2 with freq 25. Queue: z_1:14, d:16, z_2:25, a:45

Iteration 3:

Extract z_1(14) and d(16). Create z_3 with freq 30. Queue: z_2:25, z_3:30, a:45

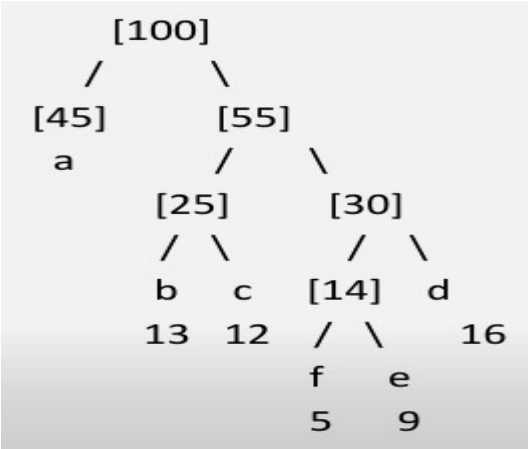
Iteration 4:

Extract z_2(25) and z_3(30). Create z_4 with freq 55. Queue: a:45, z_4:55

Iteration 5:

Extract a(45) and z_4(55). Create root with freq 100.

The resulting tree



Reading off the codewords (left = 0, right = 1):

Character	Codeword	Length	Frequency	Contribution
a	0	1	45	45
b	100	3	13	39
c	101	3	12	36
d	111	3	16	48
e	1101	4	9	36
f	1100	4	5	20

Total bits: 45+39+36+48+36+20 = 224 thousand. Compared to 300 with fixed-length (a 25.3% saving).

The most frequent character (a, 45%) gets the shortest code (1 bit). The rarest (f, 5%) gets the longest (4 bits). Exactly as intended.

Running Time

Each EXTRACT-MIN and INSERT on a binary min-heap costs O(log n).

The loop runs n−1 times.

$T(n) = O(n \log n)$

When Greedy Fails

The Coin Change Problem

Problem: Make change for amount n using coins of given denominations. Minimize the number of coins used.

Greedy strategy: Always use the largest denomination coin that doesn't exceed the remaining amount.

With denominations {1, 5, 10, 25}: Greedy works perfectly.

Change for 36: 25 + 10 + 1 = 3coins

With denominations {1, 3, 4}: Greedy fails.

Change for 6: Greedy takes 4, then 1, then 1 → 3 coins

Optimal: 3 + 3 →2coins

The greedy choice (take 4) is locally optimal but globally wrong. Why? Because the greedy choice property fails there is no optimal solution for amount 6 that includes a coin of value 4.

This problem requires Dynamic Programming.

The 0/1 Knapsack Problem

Problem: Given items with weights and values, fill a knapsack of capacity W to maximize total value. Each item is either taken or not.

Greedy strategy: Always take the item with the highest value-to-weight ratio.

Consider: Capacity = 10,

items = {(weight:6, value:7), (weight:5, value:5), (weight:5, value:5)}.

Ratios: 1.17, 1.0, 1.0.

Greedy takes weight-6 item (value 7), can only fit one weight-4 item (but there are none). Gets value 7.

Optimal: two weight-5 items, value 10. Greedy fails.

Interestingly, the Fractional Knapsack (where you can take fractions of items) is solvable by greedy.

The Greedy Algorithm Design Template

Every greedy algorithm follows this same structure:

GENERIC-GREEDY(Problem):

```
1  Sort or organize input according to the greedy criterion
2  solution = empty
3  for each element in organized order:
4  if element is compatible with current solution:
5   add element to solution
6  return solution
```

(14)Dynamic Programming

Dynamic Programming (DP) is an algorithmic technique used to solve complex problems by breaking them down into simpler subproblems.

It is essentially recursion with a memory.

Instead of recomputing the same subproblems over and over again, DP solves each subproblem exactly once and stores the result for future use.

The Two Properties

Dynamic programming applies to optimization problems exhibiting two properties.

Both are necessary. Neither alone is sufficient.

1. Optimal Substructure

A problem has optimal substructure if an optimal solution to the problem contains within it optimal solutions to subproblems.

How to prove it: Always use a cut-and-paste argument. Assume you have an optimal solution to the original problem. Identify a subproblem embedded within it. Assume for contradiction that the subproblem solution is not optimal, there exists a better one.

Cut out the suboptimal subproblem solution, paste in the better one. The overall solution improves, contradicting the assumption that the original was optimal.

Therefore the subproblem solution must have been optimal all along.

2. Overlapping Subproblems

A problem has overlapping subproblems if a recursive algorithm solving it encounters the same subproblems repeatedly not just once.

This is the property that distinguishes DP from divide and conquer.

In Merge Sort, the subproblems (sort the left half, sort the right half) are completely disjoint. The left half and right half share no elements and will never appear as subproblems of each other. Solving one gives no information useful for the other.

Memoization would give no benefit.

In DP problems, the same subproblem appears from multiple different decompositions of the original problem. Computing the optimal solution for a subproblem of size k is needed regardless of which larger problem you are solving that contains it. Solving it once and storing the result saves all future recomputations.

The test: Draw the recursion tree for the naive recursive algorithm. If any node appears more than once (if the same subproblem label appears at multiple nodes) overlapping subproblems exist and DP applies.

The Two Approaches

Once you have confirmed both properties, you implement DP in one of two equivalent ways.

Top-Down with Memoization: Write the natural recursive solution, but augment it with a memo table. Before computing a subproblem, check if its answer is already in the table. If yes, return the stored value immediately. If no, compute it, store it, then return it.

This approach is "top-down" because you start at the original problem and recurse downward into subproblems on demand.

Bottom-Up Tabulation: Identify the order in which subproblems depend on each other. Solve the smallest subproblems first, filling a table, then use those results

to solve larger subproblems. By the time you need a subproblem's result, it's already in the table.

This approach is "bottom-up" because you start from the base cases and build upward to the final answer.

Both approaches give identical results and identical asymptotic complexity. Bottom-up typically has smaller constants (no recursion overhead).

Top-down is often easier to derive initially because it follows the natural recursive structure.

Rod Cutting

A company buys long steel rods and cuts them into shorter pieces for sale. Each piece of length I inches sells for price p_i dollars. The company wants to know: given a rod of length n inches, how should it be cut to maximize total revenue?

The price table:

Length i	1	2	3	4	5	6	7	8	9	10
Price p_i	1	5	8	9	10	17	17	20	24	30

Notice that selling a rod uncut is not always optimal. A rod of length 4 sells for 9 uncut. But cutting it into two pieces of length 2 gives 5 + 5 = 10. Cutting into four pieces of length 1 gives only 4.

Why is this hard? For a rod of length n there are 2n-1 ways to cut it each of the n-1 positions either has a cut or does not. For n = 30that is over 500 million possibilities.

Optimal Substructure of Rod Cutting

The key observation: Make a first cut of length i (where $1 \leq i \leq n$). You receive p_i for that piece. The remaining rod has length n- i. Solve that remaining rod optimally. The total revenue for the optimal cutting of a length-n rod is:

$r(n) = \max$ over i from 1 to n of $\{ p_i + r(n - i) \}$

with $r(0) = 0$.

Proving optimal substructure: Suppose you have an optimal cutting of a length-n rod achieving revenue r(n). Look at the first piece, say it has length i. The remaining n- i inches must be cut optimally, achieving r(n- i).

Why? If there were a better cutting of the remaining n- i inches achieving revenue r' > r(n- i), then we could use that instead, getting total revenue $p_i + r' > p_i + r(n- i) = r(n)$. This contradicts the assumption that r(n) was optimal. So the remainder must be cut optimally.

Overlapping subproblems:

Consider computing r(4).

We try i = 1 and need r(3).

We try i = 2 and need r(2).

We try i = 3 and need r(1).

Now when we computer(3),

we need r(2) and r(1) again.

When we computer(2), we need r(1) again.

r(1) is computed multiple times, r(2) multiple times.

For large n this redundancy is exponential.

Both properties confirmed. DP applies.

The Naive Recursive Algorithm

CutRod(p, n):

if n == 0: return 0

q =-∞

for i = 1 to n:

q = max(q, p[i] + CutRod(p, n- i))

return q

Recurrence for running time:

T(0) = 1 T(n) = 1 + T(0) + T(1) + ... + T(n-1)

Solving: T(n) = 2n. **Exponential.**

For n = 30 this is over one billion calls.

Recursion tree for n = 4:

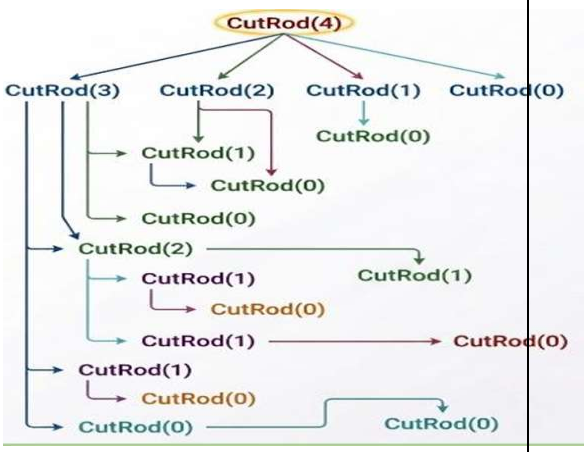
CutRod(2) appears twice.

CutRod(1) appears three times.

CutRod(0) appears four times. For n = 10,

CutRod(0) would appear hundreds of times.

This is the overlapping subproblems property made visible.



Top-Down DP

Add a memo table. Every subproblem is computed at most once.

MemoizedCutRod(p, n):

r[0..n] = all -∞ ← -∞ signals "not yet computed"

return MemoizedCutRodAux(p, n, r)

MemoizedCutRodAux(p, n, r):

if r[n] >= 0:

return r[n] ← already computed, return immediately

if n == 0:

q = 0

else:

q = -∞

for i = 1 to n:

q = max(q, p[i] + MemoizedCutRodAux(p, n - i, r))

r[n] = q ← store before returning

return q

Time complexity: Each subproblem r[0], r[1], ..., r[n] is computed exactly once. Computing r[j] requires the inner loop running j times O(j) work. Total:

$$\sum(j=0 \text{ to } n)\ j = n(n+1)/2 = \Theta(n^2)$$

From $\Theta(2n)$ to $\Theta(n^2)$.

This is the fundamental transformation dynamic programming achieves, exponential to polynomial by eliminating redundant computation.

Bottom-Up DP

Solve subproblems in increasing order of size. When computing r[j], all of r[0] through r[j-1] are already in the table.

BottomUpCutRod(p, n):

r[0..n]

r[0] = 0

for j = 1 to n:

q =-∞

for i = 1 to j:

q = max(q, p[i] + r[j- i])

r[j] = q

return r[n]

No recursion. No stack. Same $\Theta(n^2)$ time, same $\Theta(n)$ space. Typically faster in practice due to better cache behavior and no function call overhead.

Full trace for n = 4:

r[0] = 0.

j = 1: i=1: p[1] + r[0] = 1 + 0 = 1. r[1] = 1.

j = 2: i=1: p[1] + r[1] = 1 + 1 = 2. i=2: p[2] + r[0] = 5 + 0 = 5. r[2] = 5.

j = 3: i=1: p[1] + r[2] = 1 + 5 = 6. i=2: p[2] + r[1] = 5 + 1 = 6. i=3: p[3] + r[0] = 8 + 0 = 8. r[3] = 8.

j = 4: i=1: p[1] + r[3] = 1 + 8 = 9. i=2: p[2] + r[2] = 5 + 5 = 10. i=3: p[3] +

r[1] = 8 + 1 = 9. i=4: p[4] + r[0] = 9 + 0 = 9. r[4] = 10.

Optimal revenue for a length-4 rod is 10 (achieved by cutting into two pieces of length 2).

Reconstructing the Solution

The table gives us the optimal revenue but not the actual cuts. To recover the cuts we store an additional table s[j] recording the length of the first piece cut when solving a rod of length j.

ExtendedBottomUpCutRod(p, n):

r[0..n], s[1..n]

r[0] = 0

for j = 1 to n:

q = -∞

for i = 1 to j:

if p[i] + r[j - i] > q:

q = p[i] + r[j - i]

s[j] = i ← record the first cut

r[j] = q

return r, s

Reconstructing the Solution

PrintCutRodSolution(p, n):

(r, s) = ExtendedBottomUpCutRod(p, n)

while n > 0:

print s[n] ← this piece

n = n - s[n] ← remaining rod length

Trace for n = 4:

From the computation above: s[1] = 1 (cut a piece of length 1 from a rod of length 1).

s[2] = 2 (cut a piece of length 2 from a rod of length 2 — don't cut).

s[3] = 3 (cut a piece of length 3 from a rod of length 3 — don't cut).

s[4] = 2 (cut a piece of length 2 first from a rod of length 4).

PrintCutRodSolution for n = 4: Print s[4] = 2. Remaining n = 4 - 2 = 2. Print s[2] = 2. Remaining n = 2 - 2 = 0. Stop.

Solution: two pieces of length 2. Revenue: 5 + 5 = 10.

This two-phase pattern (compute optimal value first, backtrack through stored choices to reconstruct the solution) is universal across all DP problems.

The DP Design Process

Step back and observe the process we followed. It is the same process for every DP problem:

Step 1 Identify the subproblem. What is the right way to break the original problem into smaller instances of the same problem? For rod cutting: a rod of length j is a subproblem of any rod of length > j.

Step 2 Prove optimal substructure. Use the cut-and-paste argument. Show that an optimal solution to the whole problem contains optimal solutions to the subproblems it uses.

Step 3 Write the recurrence. Express the optimal value for a problem of size n in terms of optimal values for smaller problems. For rod cutting: r(n) = max over i of { p[i] + r(n-i) }.

Step 4 Identify the computation order. What does each table entry depend on? Fill the table so dependencies are resolved before they are needed. For rod cutting: r[j] depends on r[0] through r[j-1] — fill left to right.

Step 5 Implement and store choices. Compute the table. Store the choice made at each step in a separate table for reconstruction.

Step 6 Reconstruct the solution. Backtrack through the stored choices from the final answer to the base case.

Subsequence

A subsequence of a sequence X is obtained by deleting zero or more elements from X without disturbing the relative order of the remaining elements.

If X = ⟨A, B, C, B, D, A, B⟩ then:

⟨A, B, D, A⟩ is a subsequence, delete positions 3, 5, 7.

⟨B, C, D, B⟩ is a subsequence, delete positions 1, 5, 6.

⟨B, A, C⟩ is NOT a subsequence, order is violated. B comes before A in X

but after C, so you cannot have B then A then C while preserving order.

A common subsequence of two sequences X and Y is a sequence that is simultaneously a subsequence of X and a subsequence of Y.

Longest Common Subsequence

The Longest Common Subsequence is the common subsequence of maximum length. There may be multiple LCS of the same maximum length.

Example: X = ⟨A, B, C, B, D, A, B⟩ Y = ⟨B, D, C, A, B, A⟩

⟨B, C, A⟩ is a common subsequence of length 3. ⟨B, D, A, B⟩ is a common subsequence of length 4. ⟨B, C, B, A⟩ is a common subsequence of length 4.

Is there a common subsequence of length 5? No, we will prove the LCS

has length 4.

LCS

```
LCSLength(X, Y, m, n):
  c[0..m][0..n]                                          else if c[i-1][j] >= c[i][j-1]:
  b[1..m][1..n]        ← direction table for reconstruction        c[i][j] = c[i-1][j]
  for i = 0 to m: c[i][0] = 0    ← base cases: empty Y             b[i][j] = "↑“ ← skip x_i:
                                                                          came from above
  for j = 0 to n: c[0][j] = 0    ← base cases: empty X              else:
  for i = 1 to m:                                                        c[i][j] = c[i][j-1]
   for j = 1 to n:                                                b[i][j] = "←" ← skip y_j: came
                                                                          from left
      if X[i] == Y[j]:                                                 return c[m][n], b
       c[i][j] = c[i-1][j-1] + 1
```

b[i][j] = " " ← match: came from diagonal

Time: Θ(mn) fill an m×n table with O(1) work per cell. Space: Θ(mn) the c and b tables.

LCS

LCS GRID FILLING RULES (Start at top-left corner):

1. IF LETTERS MATCH (X[i] == Y[j]):

Look at the cell DIAGONALLY UP-LEFT ( ).

Add 1 to that value and write it in the current cell.

2. IF LETTERS DO NOT MATCH (X[i] != Y[j]):

Look at the cell directly UP (↑) and directly LEFT (←).

Take the HIGHEST number between the two and write it in the current cell.

Example

X = ⟨A, B, C, B, D, A, B⟩ — m = 7 Y = ⟨B, D, C, A, B, A⟩ — n = 6

Fill the c table row by row. Rows are indexed by X characters (A,B,C,B,D,A,B), columns by Y characters (B,D,C,A,B,A). Row 0 and column 0 are all zeros.

	""	B	D	C	A	B	A
""	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	4

c[7,6] = 4. The LCS has length 4.

LCS

1.If letters match (x[I] == y[J]):

This letter is part of the answer. Save it.

Move DIAGONAL UP-LEFT ().

2. If letters do not match and neighbors are different:

Look at the cell directly UP and the cell directly LEFT.

Move toward the HIGHER number.

3. If letters do not match and neighbors are tied (same value):

Both directions are correct.

Move UP to find one sequence, or move LEFT to find another.

Reconstructing the LCS: Follow the b table from b[m,n] back toward the top-left corner. Every "" arrow means the characters at that position matched and

are part of the LCS.

```
PrintLCS(b, X, i, j):
if i == 0 or j == 0:
return                ← base case: nothing to print
if b[i][j] == " ":
PrintLCS(b, X, i-1, j-1)
print X[i]            ← this character is in the LCS
else if b[i][j] == "↑":
PrintLCS(b, X, i-1, j)
else:
PrintLCS(b, X, i, j-1)
```

Space Optimization

The full c and b tables use Θ(mn) space. If you only need the LCS length (not the actual sequence) you can reduce to Θ(n) space.

The key observation: c[i,j] depends only on c[i-1,j-1], c[i-1,j], and c[i,j

1]. When filling row i, you only need row i-1 and the current row i. So

keep only two rows at a time.

Space: Θ(n). Time remains Θ(mn).

The trade-off: you lose the ability to reconstruct the LCS from b arrows since you did not store the full b table. If reconstruction is needed, keep the full tables. If only the length is needed, use this optimization.

(15)Graph Algorithms

Graph: A graph G = (V, E) consists of a set of vertices (or nodes) V and a set of edges E, where each edge connects a pair of vertices.

Directed vs undirected. In an undirected graph, an edge {u, v} represents a connection with no direction. In a directed graph (digraph), an edge (u, v) has a direction.

Weighted vs unweighted. In an unweighted graph, edges simply exist or do not. In a weighted graph, each edge has an associated numerical weight (representing distance, cost, capacity, or time). A road network is naturally weighted by distance or travel time.

Key terminology :

The degree of a vertex in an undirected graph is the number of edges incident to it. In a directed graph we distinguish in-degree (edges coming in) and out-degree (edges going out).

A **path** is a sequence of vertices where consecutive vertices are connected by an edge. A simple path has no repeated vertices.

A **cycle** is a path that returns to its starting vertex. A graph with no cycles is called acyclic.

A graph is **connected** (undirected case) if there is a path between every pair of vertices.

A directed graph is **strongly connected** if there is a directed path from every vertex to every other vertex.

Representing Graphs in Memory

There are two standard representations, and the choice between them has real complexity consequences.

1.AdjacencyMatrix

An n×n matrix A where A[i][j]=1(or the edge weight) if there is an edge from vertex I to vertex j, and 0(or∞)other wise.

Graph with vertices{1,2,3,4}and edges (1,2),(1,3),(2,4),(3,4):

	1	2	3	4
1	0	1	1	0
2	0	0	0	1
3	0	0	0	1
4	0	0	0	0

Space: $\Theta(V^2)$ regardless of how many edges actually exist.

Checking if edge (u,v) exists: $\Theta(1)$ direct lookup.

Finding all neighbors of v: $\Theta(V)$ must scan an entire row, even if v has only one neighbor.

For undirected graphs, the matrix is symmetric: A[i][j] = A[j][i].

2.AdjacencyList

For each vertex, store a list of its neighbors.

Same graph:

1:→2,3

2:→4

3:→4

4:(empty)

Space :$\Theta(V+E)$proportional to the actual number of edges, plus one list header per vertex.

Checking if edge (u,v) exists: O(degree(u))must scan u's list.

Finding all neighbors of v: Θ(degree(v)) exactly the work needed, no wasted scanning.

Adjacency list is preferred for sparse graphs graphs where E is much smaller than V^2.

Adjacency matrix is preferred for dense graphs (E close to V^2), or when constant-time edge-existence queries dominate, or for certain algebraic graph algorithms (computing graph powers via matrix multiplication, for instance).

Breadth-First Search

Breadth-First Search explores a graph in waves

It visits all vertices at distance 1 from the source before any vertex atmdistance 2, all vertices at distance 2 before any at distance 3, and so on.

This wave-like exploration is exactly what gives BFS its most important property: in an unweighted graph, BFS computes shortest paths from the source to every reachable vertex.

1. BFS(G, s):

2.for each vertex v in G.V:

3. color[v] = WHITE ←not yet discovered

4. dist[v] = ∞

5. parent[v] = NIL

6. color[s] = GRAY ←discovered, not yet finished

7. dist[s] = 0

8. Q=empty queue

9. Enqueue(Q, s)

10. while Q is not empty:

11. u = Dequeue(Q)

12. for each v in G.Adj[u]:

13 if color[v] == WHITE:

```
14   color[v] = GRAY
15   dist[v] = dist[u] + 1
16   parent[v] = u
17  Enqueue(Q, v)
18   color[u] = BLACK        ← finished
```

Breadth-First Search

The three colors: WHITE means undiscovered. GRAY means discovered but its neighbors have not all been examined yet — it is "on the frontier." BLACK means finished discovered and all neighbors examined.

Why FIFO Order Gives Shortest Paths

The queue is FIFO (first in, first out). This means vertices are processed in the exact order they were discovered.

Claim: When a vertex v is dequeued, dist[v] correctly holds the shortest-path distance (in number of edges) from s to v.

Time Complexity

Each vertex is enqueued and dequeued exactly once: O(V).

Each vertex's adjacency list is scanned exactly once, across the entire algorithm and the total size of all adjacency lists is Θ(E). So the total work in the inner loop, summed across all dequeues, is Θ(E).

Total: Θ(V + E).

The BFS Tree and Shortest Paths Reconstruction

The parent pointers set during BFS form a tree rooted at s the BFS tree.

The path from any vertex v back to s, following parent pointers, is a shortest path in the original graph (in terms of edge count).

Breadth-First Search

```
Print Shortest Path(G, s, v):
if v == s:
```

print s

elif parent[v] == NIL:

print "no path exists"

else:

Print Shortest Path(G, s, parent[v])

print v

Example

Graph: vertices {1,2,3,4,5,6}, edges (1,2),(1,3),(2,4),(3,4),(3,5),(4,6),(5,6). Source s=1.

Initial: color all WHITE, dist all ∞.

color[1]=GRAY, dist[1]=0. Enqueue 1. Q=[1].

Dequeue 1. Neighbors: 2, 3.

2 is WHITE → color[2]=GRAY, dist[2]=1, parent[2]=1. Enqueue 2.

3 is WHITE → color[3]=GRAY, dist[3]=1, parent[3]=1. Enqueue 3.

color[1]=BLACK. Q=[2,3].

Dequeue 2. Neighbors: 1, 4.

1 is BLACK → skip.

4 is WHITE → color[4]=GRAY, dist[4]=2, parent[4]=2. Enqueue 4.

color[2]=BLACK. Q=[3,4].

Dequeue 3. Neighbors: 1, 4, 5.

1 is BLACK → skip.

4 is GRAY → skip (already discovered).

5 is WHITE → color[5]=GRAY, dist[5]=2, parent[5]=3. Enqueue 5.

color[3]=BLACK. Q=[4,5].

Dequeue 4. Neighbors: 2, 3, 6.

2,3 BLACK → skip.

6 is WHITE → color[6]=GRAY, dist[6]=3, parent[6]=4. Enqueue 6.

color[4]=BLACK. Q=[5,6].

Dequeue 5. Neighbors: 3, 6.

3 BLACK → skip. 6 GRAY → skip.

color[5]=BLACK. Q=[6].

Dequeue 6. Neighbors: 4, 5. Both BLACK.

color[6]=BLACK. Q=[].

Final distances: dist = [0,1,1,2,2,3] for vertices 1-6. Shortest path to vertex 6: 1 → 2 → 4 → 6, or equivalently 1 → 3 → 5 → 6 if that had been found first — both have length 3, and the BFS tree picks one based on traversal order, but both are valid shortest paths.

Depth-First Search

Where BFS explores broadly (level by level) Depth-First Search explores deeply: it follows a path as far as possible before backtracking.

This is the natural recursive exploration strategy, and it reveals different structural information about the graph than BFS does.

1. DFS(G):

2. for each vertex u in G.V:

3. color[u] = WHITE

4. parent[u] = NIL

5. time = 0

6. for each vertex u in G.V:

7. if color[u] == WHITE:

8. DFS-Visit(G, u)

9. DFS-Visit(G, u):

10. time = time + 1

11. discovery[u] = time ← timestamp: when u was first reached

12. color[u] = GRAY

13. for each v in G.Adj[u]:

14. if color[v] == WHITE:

15. parent[v] = u

16. DFS-Visit(G, v)

17. color[u] = BLACK

18. time = time + 1

19. finish[u] = time ← timestamp: when u's exploration completed

Depth-First Search

The outer loop matters.

Unlike BFS, which is typically run from a single source, DFS as defined here visits the entire graph (including vertices unreachable from any particular starting point) by restarting from any unvisited vertex.

Edge Classification

DFS classifies every edge (u,v) of the graph into one of four types, based on the colors and timestamps:

Tree edge: (u,v) is an edge in the DFS forest, v was first discovered via this edge (v was WHITE when (u,v) was explored, and parent[v]=u was set).

Back edge: (u,v) connects u to an ancestor v in the DFS tree, v is GRAY when (u,v) is explored.

Forward edge: (u,v) connects u to a descendant v that has already been fully finished (BLACK) these occur only in directed graphs.

Cross edge: all other edges connecting vertices in different DFS trees, or in the same tree but neither an ancestor/descendant relationship (BLACK, and finish[v] < discovery[u] would not hold in a tree relationship).

Time Complexity

The outer loop visits each vertex once: $\Theta(V)$.

DFS-Visit, across all calls, examines every entry in every adjacency list exactly once: Θ(E).

Total: Θ(V + E). Same as BFS

Both are linear-time traversals; they differ only in exploration order and in what information they extract along the way.

Example

Directed graph: vertices {1,2,3,4,5}, edges

(1,2),(1,3),(2,4),(3,4),(4,5),(5,3).

Start DFS-Visit(1). time=1. discovery[1]=1. color[1]=GRAY.

Neighbors of 1: 2, 3.

2 is WHITE → parent[2]=1, DFS-Visit(2).

time=2. discovery[2]=2. color[2]=GRAY.

Neighbors of 2: 4.

4 is WHITE → parent[4]=2, DFS-Visit(4).

time=3. discovery[4]=3. color[4]=GRAY.

Neighbors of 4: 5.

5 is WHITE → parent[5]=4, DFS-Visit(5).

time=4. discovery[5]=4. color[5]=GRAY.

Neighbors of 5: 3.

3 is WHITE → parent[3]=5, DFS-Visit(3).

time=5. discovery[3]=5. color[3]=GRAY.

Neighbors of 3: 4.

4 is GRAY → edge (3,4) is a BACK EDGE (4 is an ancestor of 3).

color[3]=BLACK. time=6. finish[3]=6.

color[5]=BLACK. time=7. finish[5]=7.

color[4]=BLACK. time=8. finish[4]=8.

color[2]=BLACK. time=9. finish[2]=9.

Back at 1. Next neighbor: 3.

3 is BLACK → edge (1,3): discovery[3]=5 > discovery[1]=1 and finish[3]=6 < finish[1] → 3 is a descendant of 1 already finished → FORWARD EDGE.

color[1]=BLACK. time=10. finish[1]=10.

(16)Shortest Paths

Relaxation:

For every vertex v, maintain two pieces of information:

d[v] Our current best estimate of the shortest distance from the source s to v. Initialize d[s] = 0 and d[v] = ∞ for all other v.

π[v] The predecessor of v on the current best-known path from s. Initialize π[v] = NIL.

Relaxing an edge (u,v) means asking: does going through u give a shorter path to v than what we currently know?

Relax(u, v, w):

if d[v] > d[u] + w(u,v):

d[v] = d[u] + w(u,v)

π[v] = u

Dijkstra's Algorithm

What if edges have weights, and "distance" means total weight rather than edge-count?

The natural generalization: instead of a FIFO queue (which always gives you the earliest discovered vertex), use a min-priority queue keyed by d[v] always giving you the vertex with the smallest current distance estimate.

This only works correctly if all edge weights are non-negative.

Dijkstra(G, w, s):

for each vertex v in G.V:

d[v] = ∞

π[v] = NIL

d[s] = 0

S =∅ ←set of vertices with finalized distances

Q= min-priority queue containing all vertices, keyed by d[v]

while Q is not empty:

u = ExtractMin(Q) ←vertex with smallest d[u] among Q

S =S∪{u}

for each v in G.Adj[u]:

Relax(u, v, w)

if d[v] was decreased:

DecreaseKey(Q, v, d[v])

- At each step, extract the vertex with the smallest tentative distance, declare it final, and relax all its outgoing edges possibly improving the tentative distances of its neighbors.

Example

Graph (directed, weighted): vertices {A,B,C,D,E}. Edges: A→B (4), A→C

(2), C→B (1), B→D (5), C→D (8), C→E (10), D→E (2).

Source = A.

Init: d[A]=0, all others ∞. Q = {A,B,C,D,E}.

Extract A (d=0). S={A}.

Relax A→B: d[B] = min(∞, 0+4) = 4. π[B]=A.

Relax A→C: d[C] = min(∞, 0+2) = 2. π[C]=A.

Q distances: B=4, C=2, D=∞, E=∞.

Extract C (d=2, smallest). S={A,C}.

Relax C→B: d[B] = min(4, 2+1) = 3. π[B]=C.

Relax C→D: d[D] = min(∞, 2+8) = 10. π[D]=C.

Relax C→E: d[E] = min(∞, 2+10) = 12. π[E]=C.

Q distances: B=3, D=10, E=12.

Extract B (d=3, smallest). S={A,C,B}.

Relax B→D: d[D] = min(10, 3+5) = 8. π[D]=B.

Q distances: D=8, E=12.

Extract D (d=8, smallest). S={A,C,B,D}.

Relax D→E: d[E] = min(12, 8+2) = 10. π[E]=D.

Q distances: E=10.

Final shortest distances from A: B=3, C=2, D=8, E=10.

Shortest path to E: follow π backwards. π[E]=D, π[D]=B, π[B]=C, π[C]=A.

Path: A →C→B→D→E,withcost2+1+5+2=10.

Time Complexity

The algorithm performs |V| ExtractMin operations and, across all iterations, |E| DecreaseKey operations (one potential decrease per edge relaxation).

With a binary min-heap: ExtractMin is O(log V), DecreaseKey is O(logV). Total: O((V + E) log V) = O(E log V) for connected graphs (since E ≥ V-1).

Fibonacci Heap: O(VlogV + E) Structurally faster for dense graphs, though rarely used in practice due to high constant factors.

Bellman-Ford

d[v, i] = the shortest path from s to v using at most i edges.

d[v, 0] = 0 if v=s, else ∞.

d[v, i] = min(d[v, i-1], min over edges (u,v) of {d[u, i-1] + w(u,v)})

This is a 1D sequential DP (exactly like rod cutting) except

the "size" being incremented is the edge budget i, not a rod length.

BellmanFord(G, w, s):

for each vertex v in G.V:

d[v] = ∞

π[v] = NIL

d[s] = 0

for i = 1 to |V|- 1: ←the DP"edge budget"dimension

for each edge (u,v) in G.E:

Relax(u, v, w)

for each edge (u,v) in G.E: ←one extra pass: cycle check

if d[v] > d[u] + w(u,v):

return "negative-weight cycle detected"

return d, π

Example

Graph (directed): vertices {A,B,C,D}. Edges: A→B(6), A→C(4), B→D(-3), C→B(1), C→D(8).

Source = A. |V|-1 = 3 rounds.

Init: d=[A:0, B:∞, C:∞, D:∞].

Round 1: relax all edges in some fixed order (A→B, A→C, B→D, C→B, C→D):

A→B:d[B]=min(∞,0+6)=6.

A→C:d[C]=min(∞,0+4)=4.

B→D:d[D]=min(∞,6+(-3))=3.

C→B:d[B]=min(6,4+1)=5.

C→D:d[D]=min(3,4+8)=3 (no change).

After round 1: d=[A:0, B:5, C:4, D:3].

Round 2:

A→B:min(5, 0+6)=5 (no change).

A→C:min(4, 0+4)=4 (no change).

B→D:min(3, 5+(-3))=2. d[D]=2.

C→B:min(5, 4+1)=5 (no change).

C→D:min(2, 4+8)=2 (no change).

After round 2: d=[A:0, B:5, C:4, D:2].

Round 3: All edges: no further changes. (Already converged.)

After round 3: d=[A:0, B:5, C:4, D:2]. Unchanged(stable).

Final check: all edges satisfy $d[v] \leq d[u]+w(u,v)$. No negative cycle.

Final shortest distances: B=5, C=4, D=2. Notice this is correct despite the negative edge B→D.

Time Complexity

|V|-1 rounds, each relaxing all |E| edges: O(VE). The final cycle-check pass is O(E), absorbed into the bound.

Compare to Dijkstra's O(E log V): for sparse graphs (E ≈ V), Dijkstra is O(V log V) versus Bellman-Ford's $O(V^2)$.

Dijkstra is substantially faster, but only applicable when weights are non-negative.

This is the fundamental trade-off: generality (Bellman-Ford) versus speed (Dijkstra).**Choosing the Right Algorithm**

Scenario	Algorithm	Time
Single source, non-negative weights	Dijkstra	O(E log V)
Single source, negative weights allowed, need cycle detection	Bellman-Ford	O(VE)
All pairs, dense graph or negative weights present	Floyd-Warshall	$O(V^3)$
All pairs, sparse graph, non-negative weights (after reweighting)	Johnson's algorithm	$O(V^2 \log V + VE)$

(17)Network Flow and Disjoint Sets

Flow Network

A flow network is a directed graph G = (V, E) where each edge (u,v) has a capacity c(u,v) ≥ 0 (the maximum amount that can flow through that edge. There is a designated source s (flow originates here) and a sink t (flow terminates here).

Real-World Applications:

• Data Communications.

• Logistics & Supply Chains.

• Fluid Dynamics.

A function f: $E \rightarrow \mathbb{R}\geq 0$ assigns an actual current flow value to every single edge. To be considered valid, this flow must obey two strict mathematical laws:

i. Capacity constraint: $0 \leq f(u,v) \leq c(u,v)$ for every edge. You cannot push more flow through a pipe than its capacity.

ii. Flow conservation: for every vertex v other than s and t, the total flow into v equals the total flow out of v:

$$\text{For every vertex } v \in V \setminus \{s,t\} : \quad \sum_{(u,v)\in E} f(u,v) = \sum_{(v,w)\in E} f(v,w)$$

Flow cannot accumulate or be created at intermediate vertices what comes in must go out. (s and t are exempt. s is a net source of flow, t is a net sink.)

The Maximum Flow Problem

The value of a flow |f| is the total flow leaving s (equivalently, by conservation applied network-wide, the total flow arriving at t):

$$|f| = \sum_{(s,v)\in E} f(s,v)$$

Maximum flow The maximum possible value of |f| over all valid flows f.

Residual Graphs

Given a flow f, the residual capacity of edge (u,v) is:

$$cf(u,v) = c(u,v) - f(u,v)$$

This is how much more flow could be pushed along (u,v) in the forward direction.

For every edge (u,v) carrying flow $f(u,v) > 0$, the residual graph also includes a reverse edge (v, u) with residual capacity:

$$cf(v, u) = f(u,v)$$

It represents the ability to undo flow already sent along (u,v), if we later discover a better routing, we can decrease f(u,v), which is equivalent to pushing flow backward along (v,u) in the residual graph.

This is not flow physically moving backward through a pipe, it is bookkeeping that allows the algorithm to correct earlier suboptimal choices.

The residual graph Gf consists of all edges (u,v) (original or reverse) with $cf(u,v) > 0$, labeled with their residual capacities.

Augmenting Path

An augmenting path is a path from s to t in the residual graph Gf. If such a path exists, we can increase the flow:

let cf(p) = the minimum residual capacity along the path (the bottleneck).

Push cf(p) additional units of flow along every edge of p, increasing f(u,v) by cf(p) for forward edges of p, and decreasing f(v,u) by cf(p) for reverse edges of p (which corresponds to undoing flow on the original edge (v,u)).

The Ford-Fulkerson Method

FordFulkerson(G, s, t):

for each edge (u,v) in G.E:

f(u,v) = 0

while there exists an augmenting path p from s to t in Gf:

cf(p) = min residual capacity along p

for each edge (u,v) in p:

if (u,v) is a forward edge:

f(u,v) = f(u,v) + cf(p)

else: ←(u,v) is a reverse edge, i.e., (v,u) is original

f(v,u) = f(v,u)- cf(p)

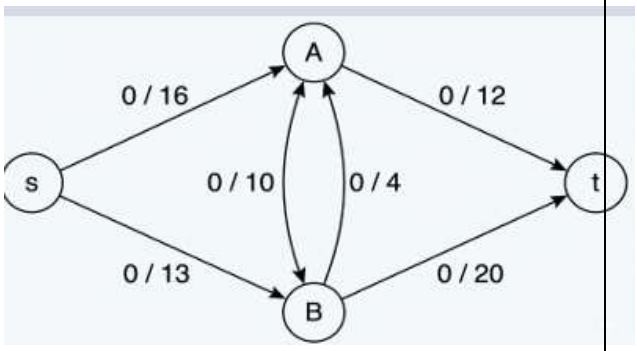
return f

This is called a method rather than an algorithm deliberately it does not specify how to find an augmenting path. Different choices (any path via DFS, shortest path via BFS) give different running times.

Example

Network: s→A (16), s→B (13), A→B (10), A→t (12), B→t (20), B→A (4).

Start with f = 0 everywhere.



Example

Iteration 1: Finding an Augmenting Path

1. Select Path:

p1 =[s-> A-> t]

2. Calculate Residual Capacities along p1:

c_f(s, A) = c(s, A)- f(s, A) = 16- 0 = 16

c_f(A, t) = c(A, t)- f(A, t) = 12- 0 = 12

3. Find Bottleneck:

c_f(p1) = min(16, 12) = 12

4. Update Flows (All Forward Hops, Add Flow):

f(s, A) = 0 + 12 = 12- f(A, t) = 0 + 12 = 12 [Saturated!]

5. Flow Value Status:

Total System Flow |f| = 0 + 12 = 12

Iteration 2: Finding an Augmenting Path

1. Select Path:

p2 =[s-> B-> t]

2. Calculate Residual Capacities along p2:

c_f(s, B) = c(s, B)- f(s, B) = 13- 0 = 13

c_f(B, t) = c(B, t)- f(B, t) = 20- 0 = 20

3. Find Bottleneck:

c_f(p2) = min(13, 20) = 13

4. Update Flows (All Forward Hops Add Flow):

f(s, B) = 0 + 13 = 13[Saturated!]

f(B, t) = 0 + 13 = 13

5. Flow Value Status:

Total System Flow |f| = 12 + 13 = 25

Iteration 3: Finding an Augmenting Path

1. Select Path:

p3 =[s-> A-> B-> t]

2. Calculate Residual Capacities along p3:

c_f(s, A) = c(s, A)- f(s, A) = 16- 12 = 4

c_f(A, B) = c(A, B)- f(A, B) = 10- 0 = 10

c_f(B, t) = c(B, t)- f(B, t) = 20- 13 = 7

3. Find Bottleneck:

c_f(p3) = min(4, 10, 7) = 4

4. UpdateFlows (All Forward Hops Add Flow):

f(s, A) = 12 + 4 = 16 [Saturated!]

f(A, B) = 0 + 4 = 4

f(B, t) = 13 + 4 = 17

5. Flow Value Status:

Total System Flow |f| = 25 + 4 = 29

Iteration 4: Termination Check

1. Evaluate Residual Capacities out of Source (s):

$c_f(s, A) = 16 - 16 = 0$

$c_f(s, B) = 13 - 13 = 0$

2. Conclusion:

Because all outgoing residual capacity from s is completely 0, no more paths can be discovered from s to t.

The algorithm officially terminates.

Edmonds-Karp

Ford-Fulkerson's running time depends entirely on how augmenting paths are chosen. With an adversarial choice of paths (always picking paths with tiny bottlenecks), the number of iterations can be proportional to the value of the flow itself (not the size of the graph). For large capacities, this is catastrophically slow.

Edmonds-Karp's fix:

Always choose the augmenting path with the fewest edges

i.e., find it via BFS on the residual graph, rather than arbitrary DFS.

The result:

With BFS-chosen augmenting paths, the number of iterations is $O(VE)$, and each BFS takes $O(E)$ giving a total running time of $O(VE^2)$.

Disjoint Sets (Union-Find)

The Disjoint Set is a specialized data structure designed to manage a collection of elements split into non-overlapping (disjoint) groups.

Every disjoint set structure must support exactly two primary operations::

Find(x) return an identifier for the set containing element x. Two elements are in the same set if and only if Find returns the same identifier for both.

Union(x,y) merge the sets containing x and y into a single set.

This sounds abstract but the application is concrete and recurring: dynamic connectivity.

Given a graph where edges are added one at a time, are two vertices u and v

currently connected?

Initialize each vertex as its own set; for each edge (u,v), if Find(u) ≠ Find(v),

they were in different components, Union them.

Representation (Forest of Trees)

Each set is represented as a tree, where every node points to its parent, and the root is the set's representative. Find(x) walks up parent pointers until reaching a root. Union(x,y) makes one root point to the other.

MakeSet(x):

parent[x] = x

rank[x] = 0

Find(x):

if parent[x] != x:

return Find(parent[x])

return x

Union(x,y):

rootX = Find(x)

rootY = Find(y)

if rootX == rootY:

return ←already in the same set

Link(rootX, rootY)

Optimization 1 (Union by Rank)

Without care, repeated unions can produce a long chain (a "linked list" tree), making Find take O(n) time**. Union by rank** prevents this: track an approximate height

(rank) for each tree, and when uniting two trees, make the root of the shorter tree

point to the root of the taller tree (keeping the combined tree's height as small as possible).

Link(x, y): ←x and y are roots

if rank[x] > rank[y]:

parent[y] = x

elif rank[x] < rank[y]:

parent[x] = y

else:

parent[y] = x

rank[x] = rank[x] + 1 ←only increases when ranks were equal

With union by rank alone, the height of any tree is O(log n) giving Find and Union both O(log n).

Optimization 2 (Path Compression)

Every time Find(x) walks up to the root, make every node on that path point directly to the root:

Find(x):

if parent[x] != x:

parent[x] = Find(parent[x]) ← recursive call returns the root; reassign parent directly to it

return parent[x]

After this, any future Find on any node that was on this path takes O(1) it points straight to the root. The tree gets flattened over time, on every

Find operation, as a side effect of simply answering the query.

The Amortized Analysis

The combined result: with both union by rank and path compression, a sequence of m Find/Union operations on n elements takes $O(m \cdot \alpha(n))$ total time, where $\alpha(n)$

is the **inverse Ackermann function.**

α(n) grows slowly for any conceivable value of n. α(n) grows so slowly that for any conceivable value of n (even n larger than the number of atoms in the observable universe) **α(n) ≤ 4.** This is the same kind of **essentially constant for all practical purposes function** as log* n from the asymptotic notation block, and it is no coincidence: both arise from the same family of extremely-slow-growing iterated functions.

The practical takeaway: Union-Find with both optimizations gives amortized O(1) per operation, for all practical purposes. This is one of the most striking results in algorithm analysis a data structure that is, in effect, as fast as array access, despite supporting dynamic merging of sets.

Example

Suppose we have a chain: 5→4→3→2→1 (parent pointers, 1 is the root), all from previous unions without compression.

Find(5): walks 5→4→3→2→1, finds root 1. With path compression, on the way back up the recursion, we set parent[5]=1, parent[4]=1, parent[3]=1, parent[2]=1 (parent[2]=1 was already true, but it gets reaffirmed).

After this single Find(5), the tree is now: 5,4,3,2 all point directly to 1. Any future Find on 2,3,4, or 5 is now O(1).

(18)String Matching

String Matching

Input: A text T of length n, and a pattern P of length m, both strings

over some alphabet Σ.

Output: Every starting position s in T ($0 \leq s \leq n-m$) such that T[s..s+m-1]

= P i.e., the pattern occurs in the text starting at position s, character for character.

Example:

T ="ABABDABACDABABCABAB“

P ="ABABCABAB".

The pattern occurs starting at position 10.

The Naïve Algorithm

The most direct approach: try every possible starting position in the text, and at each one, check character by character whether the pattern matches.

NaiveStringMatch(T, P, n, m):

for s = 0 to n- m:

if T[s..s+m-1] == P:

print "pattern found at shift s“

Expanding the comparison explicitly:

NaiveStringMatch(T, P, n, m):

for s = 0 to n- m:

j = 0

while j < m and T[s+j] == P[j]:

j = j + 1

if j == m:

print "match at shift s"

Time Complexity

In the worst case, at each of the n-m+1 starting positions, we might compare up to m characters before finding a mismatch (or confirming a match). This gives O((n m+1)·m), which for m ≤ n/2 is O(nm).

worst case

ConsiderT ="AAAAAAAAAAAAAB"and

P ="AAAAB".

At every starting position except the last, we match A's for a while before the final character mismatches (close to m comparisons wasted at nearly every shift).

This is exactly the kind of repetitive-pattern input that makes naïve matching slow in practice, and it is precisely the kind of input where KMP's cleverness pays off most.

Knuth-Morris-Pratt

By precomputing, for the pattern alone (never looking at the text), exactly how much of a partial match can be salvaged when a mismatch occurs. This precomputed information is the failure function (also called the prefix function), and it is the single new idea that makes KMP work.

The Failure Function

For pattern P of length m, define π[j] (for j = 1 to m) as:

π[j] = the length of the longest proper prefix of P[0..j-1] that is also a suffix of P[0..j-1].

Proper prefix means a prefix that is not the entire string itself.

Example

P = "ABABACA" (m=7, indices 0-6: A,B,A,B,A,C,A).

Computing the Failure Function Efficiently

The hand computation above checked every possible suffix length per position that would be $O(m^3)$ or so in total if implemented naively. The actual algorithm computes π in O(m) using a self-referential trick:

comparing the pattern against itself, reusing previously computed π values exactly the way the main matching algorithm will reuse them against the text.

This is exactly KMP's main matching logic, applied with the pattern playing the role of both text and pattern simultaneously

Example

T ="ABABABACA",P = "ABABACA"(the pattern we computedπ for above). n=9, m=7.

π =[_, 0, 0, 1, 2, 3, 0, 1] (1-indexed as computed; π[0] is undefined/unused).

j=0, i=1: T[0]='A'. P[0]='A'. Match! j=1.

j=1, i=2: T[1]='B'. P[1]='B'. Match! j=2.

j=2, i=3: T[2]='A'. P[2]='A'. Match! j=3.

j=3, i=4: T[3]='B'. P[3]='B'. Match! j=4.

j=4, i=5: T[4]='A'. P[4]='A'. Match! j=5.

j=5, i=6: T[5]='B'. P[5]='C'. Mismatch!

Fall back: j = π[5] = 3.

Retry: P[3]='B'. T[5]='B'. Match! j=4.

j=4, i=7: T[6]='A'. P[4]='A'. Match! j=5.

j=5, i=8: T[7]='C'. P[5]='C'. Match! j=6.

j=6, i=9: T[8]='A'. P[6]='A'. Match! j=7.

j=7: j==m=7. MATCH found, ending at text position i-1=8 (i.e., starting at position 8-7+1=2, 0-indexed: position 2).

Rabin-Karp

Instead of comparing characters directly, compute a numerical hash of the pattern, and a hash of every length-m substring of the text. If the hashes don't match, the substrings certainly don't match skip immediately, no character comparison needed.

If the hashes do match, then do a character-by-character check to confirm (because hash collisions are possible).

RabinKarp(T, P, n, m, d, q):

h = dm-1 mod q ← precompute once

p = 0 ← hash of the pattern

t0 = 0 ← hash of T[0..m-1]

for i = 0 to m-1:

p = (d*p + P[i]) mod q

t0 = (d*t0 + T[i]) mod q

for s = 0 to n-m:

if p == t0: ← hash match (verify with direct comparison)

if T[s..s+m-1] == P[0..m-1]:

print "match at shift s"

if s < n-m:

t0 = (d*(t0 - T[s]*h) + T[s+m]) mod q ← roll the hash forward

Rabin-Karp

Text (T) = a a a a a aab

Pattern (P) = a a a b

Character Mapping Scheme:

a =1b=2

(n = 8)

(m =4)

Step 0: Target pattern hash computation

Mapvalues for P = "a a a b": a =1, a= 1, a=1, b=2

Target Pattern Hash Code = 1 + 1 + 1 + 2 = 5

Window 0: Text Substring = "a a a a" (Indices 0 to 3)

1. Calculate Initial Window Hash:

1 + 1 + 1 + 1 = 4

2. Evaluate:

Current Window Hash = 4

Target Pattern Hash = 5

Result: 4 != 5 (Hash Mismatch)

3. Action:

Slide window forward safely without checking characters.

Window1:Text Substring = "a a a a" (Indices 1 to 4)

1. Identify Rolling Transitions:

Character leaving window: 'a' (Value = 1)

Character entering window: 'a' (Value = 1)

2. Apply Rolling Hash Formula:

New Hash = Previous Hash- Leaving Value + Entering Value- New Hash = 4- 1 + 1 = 4

3. Evaluate:

Current Window Hash = 4

Target Pattern Hash = 5

Result: 4 != 5 (Hash Mismatch)

4. Action:

Slide window forward.

Window 2: Text Substring = "a a a a" (Indices 2 to 5)

1. Rolling Calculation:

New Hash = 4 - 1 + 1 = 4

2. Evaluate:

4 != 5 (Hash Mismatch)

3. Action: Slide window forward.

Window 3: Text Substring = "a a a a" (Indices 3 to 6)

1. Rolling Calculation:

New Hash = 4 - 1 + 1 = 4

2. Evaluate:

4 != 5 (Hash Mismatch)

3. Action:

Slide window forward.

Window 4: Text Substring = "a a a b" (Indices 4 to 7)

1. Identify Rolling Transitions:

Character leaving window: 'a' (Value = 1)

Character entering window: 'b' (Value = 2)

2. Apply Rolling Hash Formula:

New Hash = 4 - 1 + 2 = 5

3. Evaluate:

Current Window Hash = 5

Target Pattern Hash = 5

Result: 5 == 5 (Match Candidate Detected!)

4. Character-by-Character Verification Step:

Text Pos 4 ('a') == Pattern Pos 0 ('a') OK

Text Pos 5 ('a') == Pattern Pos 1 ('a') OK

Text Pos 6 ('a') == Pattern Pos 2 ('a') OK

Text Pos 7 ('b') == Pattern Pos 3 ('b') OK

Perfect match found at starting index position: 4

(19)Polynomial & Matrix Calculations

Polynomial

A polynomial of degree-bound n (meaning its degree is at most n−1) is:

$A(x) = a0+a1x+a2x2+\cdots+an-1xn-1$

There are two completely different but mathematically equivalent ways to represent this same object.

Coefficient Representation: Just the vector of coefficients:

$a0\ a1\ a2\ \ldots\ an-1$

This is how we are used to writing polynomials, and how a computer stores them by default.

Point-Value Representation: A set of n point-value pairs:

(x0,y0),(x1,y1),…,(xn−1,yn−1)where yk=A(xk)

provided the n points $x0,\ldots,xn-1$are distinct.

Take a simple case: a line, A(x) = 2 + 3x (degree-bound 2, so n=2).

Coefficient form: (2, 3)

Point-value form, using points x=0 and x=1: A(0)=2, A(1)=5. So: (0,2),(1,5).

Both describe exactly the same polynomial. Given either representation, you can recover the other, this is a basic fact from algebra: n distinct points uniquely determine a degree-bound-n polynomial, and vice versa.

The Critical Insight: Different Operations Are Cheap in Different Forms

Polynomial

Operation	**Coefficient Form**	**Point-Value Form**
Evaluate at one point	O(n) via Horner's Rule	O(1)
Add two polynomials:O(n) add corresponding coefficients		O(n) add corresponding y-values
Multiply two polynomials	$O(n^2)$	O(n)

Look closely at that last row. Multiplication is dramatically cheaper in point value form. If A and B are both given as n point-value pairs at the same n points, their product's value at each point is just $y_k^A \cdot y_k^B$ multiplication per point, O(n) total.

In coefficient form, multiplication requires combining every coefficient of A with every coefficient of B $O(n^2)$.

Polynomial

We are almost always handed polynomials in coefficient form (that's how humans write them, that's how programs store them). But we want the speed of point-value multiplication. So the plan becomes:

Convert coefficient form → point-value form. This conversion is called evaluation (literally evaluating the polynomial at a chosen set of points).

Multiply in point-value form O(n), trivial.

Convert back: point-value form → coefficient form. This is called interpolation.

If steps 1 and 3 can each be done quickly, this entire pipeline beats the naïve $O(n^2)$ multiplication. The entire content of the FFT algorithm is: how do we do evaluation and interpolation fast?

Horner's Rule

The Naive Approach and Its Flaw

The obvious way to compute $A(x_0) = a_0 + a_1x_0 + a_2x_0^2 + \cdots + a_{n-1}x_0^{n-1}$

to compute each power of x_0 separately, multiply by the corresponding coefficient, and sum. Done carelessly (recomputing each power from scratch), this costs $O(n^2)$. Done with a running power variable, it's O(n) (but still wasteful in structure).

Rewrite the polynomial by factoring x_0 out repeatedly:

$A(x0)=a0+x0(a1+x0(a2+x0(\cdots+x0(an-2+x0an-1)\cdots)))$

This nested form lets you evaluate from the innermost parenthesis outward, using one multiplication and one addition per coefficient.

Horner's Rule

HORNER(A,x0)

1 y=0

2 fori=n-1downto0

3 y=a[i]+x0*y

Return y

Example

A(x)=3+2x+5x²+4x³(coefficients:a_0=3,a_1=2,a_2=5,a_3=4).Evaluateatx_o=2.

Start: y=0

i=3:y=4+2·0=4

i=2:y=5+2·4=13

i=1:y=2+2·13=28

i=0:y=3+2·28=59

Checkdirectly:3+2(2)+5(4)+4(8)=3+4+20+32=59

Horner's Rule

Running Time

One pass through n coefficients, constant work per step.

T(n) =Θ(n)

This is optimal you cannot evaluate a degree-bound-n polynomial at one point in less than Θ(n), since you must look at every coefficient.

Multiplying Polynomials

If A(x) and B(x) both have degree-bound n, their product C(x) = A(x)B(x) has degree-bound 2n−1, with coefficients given by the convolution:

$$c_k = \sum_{j=0}^{k} a_j \, b_{k-j}$$

This is exactly what you do multiplying polynomials by hand: every term of A multiplies every term of B, and you collect terms by matching exponents.

NAIVE-POLY-MULTIPLY(A, B)

```
1  let C be a new array of length 2n-1, initialized to 0
2  for k = 0 to 2n-2
3      for j = max(0, k-n+1) to min(k, n-1)
4          c[k] = c[k] + a[j]*b[k-j]
5  return C
```

Example

A(x) = 1 + 2x (n=2), B(x) = 3 + x (n=2). Product should have degree-bound 3.

$c_0 = a_0b_0 = 1 \cdot 3 = 3$

$c_1 = a_0b_1 + a_1b_0 = 1 \cdot 1 + 2 \cdot 3 = 1+6 = 7$

$c_2 = a_1b_1 = 2 \cdot 1 = 2$

$C(x) = 3 + 7x + 2x^2$. Check directly: $(1+2x)(3+x) = 3 + x + 6x + 2x^2 = 3+7x+2x^2$

Running Time

This is the target to beat.

$T(n0 = \Theta(n2)$

Fast Fourier Transform (FFT)

Here's the trap many might fall into: "I'll just evaluate A and B at n arbitrary points using Horner's Rule, multiply pointwise, and interpolate back." Sounds promising but Horner's Rule is O(n) per point, and you need n points, giving $O(n^2)$ total. No improvement. The naïve approach to evaluation is itself too slow.

The breakthrough is realizing that if you choose the n evaluation points cleverly (not arbitrarily) evaluation at all n points simultaneously can be done in much less than $O(n^2)$ total.

FFT (Fast Fourier Transform) evaluates the polynomial at the n complex nth roots of unity the n complex solutions to $\omega n = 1$ .These points are spaced evenly around the unit circle in the complex plane.

The halving property: The n complex nth roots of unity come in pairs related

by negation (ω and $-\omega$are both nth roots of unity when n is even).This symmetry means: evaluating a degree-bound-n polynomial at all n roots of unity can be reduced to evaluating two degree-bound-(n/2) polynomials at the n/2 squared roots of unity — which are themselves the (n/2)th roots of unity.

In other words: the problem of size n reduces to two subproblems of size n/2, exactly the divide-and-conquer structure, but now applied to evaluation rather than sorting.

FFT

The Recurrence

Splitting costs Θ(n) work (separating even-indexed and odd-indexed coefficients, then combining results), plus two recursive calls on half the size:

$T(n) = 2T(n/2) + \Theta(n)$

It's the exact same recurrence shape as Merge Sort.

Apply the Master Theorem:

a=2, b=2, f(n)=Θ(n)

Compare n(logba) = n($\log_2 2$) = n^1 with f(n)=n they match (Case 2 of Master

Theorem)

Therefore:

$T(n) = \Theta(n \log n)$

Interpolation

Converting back from point-value form to coefficient form is called the Inverse FFT.

Remarkably, it uses almost the identical algorithm as the forward FFT just evaluating at the inverse roots of unity and scaling by 1/n at the end. Same recurrence, same Θ(n log n) running time.

The Complete Pipeline

Step 1: A, B in coefficient form

↓ FFT, each takes Θ(n log n)

Step 2: A, B in point-value form (at the n roots of unity)

↓ multiply pointwise, Θ(n)

Step 3: Product in point-value form

↓ Inverse FFT, Θ(n log n)

Step 4: Product in coefficient form — the final answer

Total Running Time for Polynomial Multiplication via FFT

$T(n) = \Theta(n \log n)$

Compare this to the naive $\Theta(n^2)$ from before.

For n=1,000,000: naive needs 10^{12} operations;

FFT needs roughly 2×10^7 a difference of five orders of magnitude.

This is why FFT underlies real-time audio processing, image compression, and large-integer multiplication in practice.

Matrix Multiplication

For two n×n matrices A and B, the product C = AB is defined entrywise as:

$$c_{ij} = \sum_{k=1}^{n} a_{ik}\, b_{kj}$$

SQUARE-MATRIX-MULTIPLY(A, B)

1 n=A.rows

2 let C be a newn×n matrix

3 for i =1 to n

4for j = 1 to n

5 c[i,j] = 0

6 for k = 1 to n

7 c[i,j] = c[i,j] + a[i,k]*b[k,j]

8 return C

Matrix Multiplication

Three nested loops, each running n times:

$T(n) = \Theta(n3)$

This is your baseline directly analogous to naive $\Theta(n^2)$ polynomial multiplication. The question, just as before, is: can a smarter algebraic restructuring beat this?

First Attempt: Naive Divide-and-Conquer (Doesn't Help)

Split each n×n matrix into four (n/2)×(n/2) blocks:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

Standard block matrix multiplication gives:

$C11 = A11B11 + A12B21, C12 = A11B12 + A12B22$

$C21 = A21B11 + A22B21, C22 = A21B12 + A22B22$

Computing all four quadrants requires 8 multiplications of (n/2)×(n/2) submatrices, plus $\Theta(n^2)$ additions to combine them. This gives the recurrence:

$T(n) = 8T(n/2) + \Theta(n2)$

Apply the Master Theorem: a=8, b=2, $f(n)=\Theta(n^2)$. Compare $n(\log_2 8)= n^3$

with $f(n)=n^2$ n^3 dominates (Case 1):

$T(n) = \Theta(n3)$

Matrix Multiplication

No improvement. Just splitting the matrix and recursing naively gives you

back exactly what you started with

Strassen's Insight: Cut 8 Multiplications Down to 7

Strassen discovered that through careful algebraic manipulation combining the submatrices in specific weighted sums before multiplying you can compute all four quadrants of C using only 7 matrix multiplications instead of 8, at the cost of a few extra (but cheap) additions and subtractions. The exact formulas define 7 intermediate products P_1 through P_7, each a single multiplication of sums/differences

of submatrices, such that C_{11}, C_{12}, C_{21}, C_{22} can each be written as a sum or difference of a subset of these seven P terms.

The crucial point: multiplication of n×n matrices is far more expensive than addition (Θ(nsomething) vs Θ(n^2)), so trading one multiplication for several additions is a genuine win when n is large.

The New Recurrence

$T(n) = 7T(n/2) + \Theta(n2)$

Apply the Master Theorem: a=7, b=2, f(n)=Θ(n^2). Compare $n(\log_2 7) \approx n2.807$ with

f(n)=n^2 — $n(\log_2 7)$ dominates since $\log_2 7 > 2$ (Case 1):

$T(n) = \Theta(n\log_2 7) \approx \Theta(n2.81)$

This is a genuine asymptotic improvement over Θ(n^3) purely from algebraic restructuring, with no change to the underlying computational model.

Matrix Multiplication

Putting the Two Halves Side by Side

	Naive	Smart Approach	Recurrence	Result
Polynomials:	Θ (n^2) multiply	Strassen's algorithm	T(n)=2T(n/2)+Θ(n)	Θ (n log n)
Matrices	Θ (n^3) multiply	FFT-based multiply	T(n)=7T(n/2)+Θ(n^2)	Θ (n2.81)